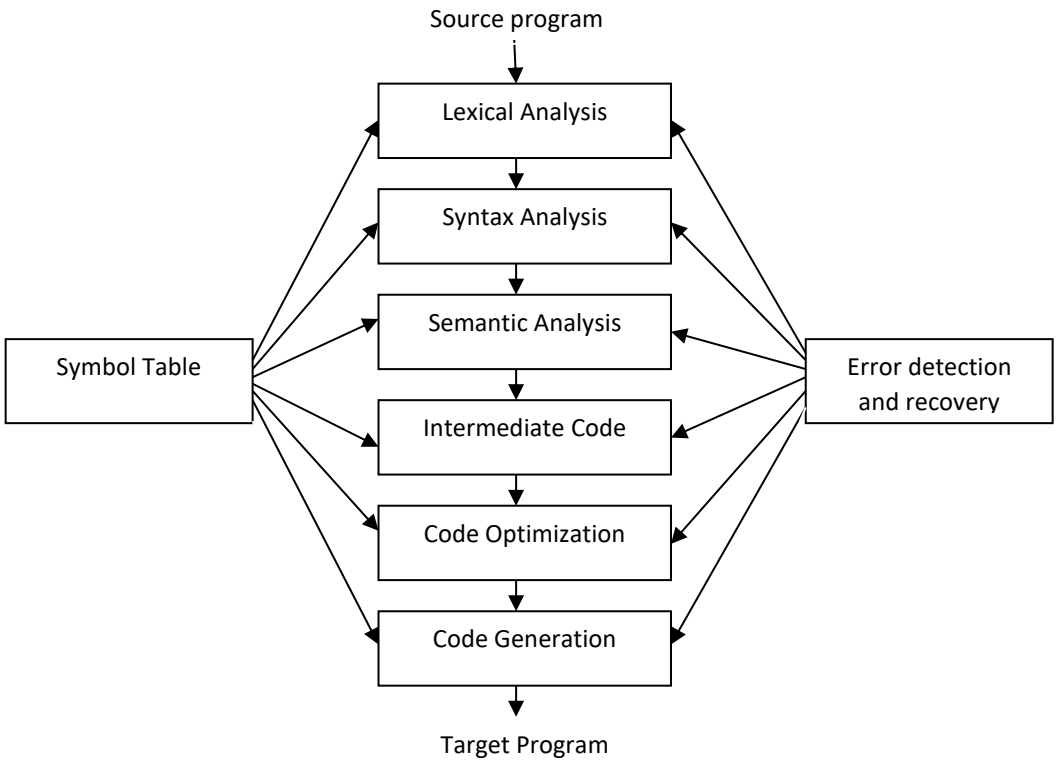
Lab 2 Study Phases of compiler.



Phases of Compiler

Examples

Explain phases of compiler with output of each phase

1. **Position=initial + rate * 60**

1. Lexical Analysis (Scanning)

Function: Converts the input program into a stream of tokens.
Output: Tokens

Token Type	Lexeme
IDENTIFER 1	Position
= OPERATOR	=
IDENTIFIER 2	initial
+ OPERATOR	+
IDENTIFIER 3	rate
* OPERATOR	*
NUMBER	60

2. Syntax Analysis (Parsing)

Function: Builds the parse tree using grammar rules.
Output: Abstract syntax Tree

```
     =
   /  \
ID1   +
       / \
    ID2  *
          / \
       ID 3  60
```

3. Semantic Analysis

Function: Performs type checking and ensures semantic validity.
Output:

```
     =
   /  \
ID1   +
       / \
    ID2  *
          / \
       ID 3  into real(60)
```

Symbol Table

Name	Type
Position	float
initial	float
rate	float

All operands are type-compatible.

4. Intermediate Code Generation

Function: Converts AST to 3-address code.
Output:
t1=into real(60)
t2= ID3 * 60
t3 = ID2 + t2
ID1 = t3

5. Code Optimization

Function: Optimizes intermediate code.
Optimized TAC:
t1 = ID3 * 60.0
ID1 = ID2 + t1

6. Target Code Generation

Function: Converts optimized TAC to assembly code.
Output:
MOV ID3, R1
MUL #60.0, R1
MOV ID2, R2
ADD R1, R2

MOV R2, ID1

2. **a = a + b * c * 2**

There are mainly two parts of compilation process.

Analysis Phase: Analysis part is divided into three sub parts,

I. Lexical analysis
II. Syntax analysis
III. Semantic analysis

Lexical analysis:

- Lexical analysis is also called linear analysis or scanning.
- Lexical analyzer reads the source program and then it is broken into stream of units. Such units are called token.
- Then it classifies the units into different lexical classes. E.g. id's, constants, keyword etc...and enters them into different tables.
- For example, in lexical analysis the assignment statement **a: = a + b * c * 2** would be grouped into the following tokens:

a	Identifier 1
=	Assignment sign
a	Identifier 1
+	The plus sign
b	Identifier 2
*	Multiplication sign
c	Identifier 3
*	Multiplication sign
2	Number 2

Syntax Analysis:

- Syntax analysis is also called hierarchical analysis or parsing.
- The syntax analyzer checks each line of the code and spots every tiny mistake that the programmer has committed while typing the code.
- If code is error free then syntax analyzer generates the tree.

```
        =
      /   \
    ID1    +
          / \
        ID1   *
             / \
            *   2
           / \
         ID2 ID3
```

Semantic analysis:

- Semantic analyzer determines the meaning of a source string.
- For example matching of parenthesis in the expression, or matching of if..else statement or performing arithmetic operation that are type compatible, or checking the scope of operation.

=

```
 /     \
ID1      +
    /   \
   ID1   *
        /  \
       *   into real(2)
      / \
    ID2 ID3
```

Synthesis phase: synthesis part is divided into three sub parts,

I. Intermediate code generation
II. Code optimization
III. Code generation

Intermediate code generation:

- The intermediate representation should have two important properties, it should be easy to produce and easy to translate into target program.
- We consider intermediate form called "three address code".
- Three address code consist of a sequence of instruction, each of which has at most three operands.
- The source program might appear in three address code as,
  t1=id2*id3
  t2=into real(2)
  t3=t1*t2
  t4=id1+t3
  id1=t4

Code optimization:

- The code optimization phase attempt to improve the intermediate code.
- This is necessary to have a faster executing code or less consumption of memory.
- Thus by optimizing the code the overall running time of a target program can be improved.

```
t1= id2 * id3
t2= t1 * 2.0
id1 = id1 + t2
```

Code generation:

- In code generation phase the target code gets generated. The intermediate code instructions are translated into sequence of machine instruction.

```
MOV id3, R1
MOV id2, R2
MUL R2, R1
MUL #2.0, R1
MOV id1, R3
ADD R3, R1
MOV R1, id1
```

Symbol Table

- A **symbol table** is a data structure used by a language translator such as a compiler or interpreter.

- It is used to store names encountered in the source program, along with the relevant attributes for those names.
- Information about following entities is stored in the symbol table.
  - ✓ Variable/Identifier
  - ✓ Procedure/function
  - ✓ Keyword
  - ✓ Constant
  - ✓ Class name
  - ✓ Label name

3. F=(9/5)*C+32

1. Lexical Analysis (Scanning)

Function: Converts the input program into a stream of tokens.
Output: Tokens

Token Type	Lexeme
IDENTIFIER 1	F
= OPERATOR	=
(BRACKET	(
NUMBER	9
/ OPERATOR	/
NUMBER	5
) BRACKET	)
* OPERATOR	*
IDENTIFIER 2	C
+ OPERATOR	+
NUMBER	32

2. Syntax Analysis (Parsing)

Function: Builds the parse tree using grammar rules.
Output: Abstract Syntax Tree (AST)

```
      =
     / \
   ID1  +
       / \
      *   32
     / \
    /   ID2
   / \
  9   5
```

3. Semantic Analysis

Function: Performs type checking and ensures semantic validation.
Output:

```
          =
         / \
      ID1   +
           / \
          *   into real(32)
         / \
        /   ID2
       / \
Into real(9)  into real(5)
```

Identifier	Type
F	Float
C	float

All operands are type-compatible.

4. Intermediate Code Generation (ICG)

Function: Converts AST to 3-address code.
Output (TAC):
t1=into real(9)
t2=into real(5)
t3=t1/t2
t4 = t3 * ID2
t5=into real(32)
t6 = t4 + t5
ID1 = t6

5. Code Optimization

Function: Optimizes intermediate code.
Optimized TAC:
t1 = 1.8 * ID2
ID1 = t1 + 32.0

6. Target Code Generation

Function: Converts optimized TAC to assembly code.
Output (Pseudo Assembly):
MOV ID2, R1
MUL #1.8, R1
ADD #32.0, R1
MOV R1, ID1

4. R=(b*b-4.0*a*c)/(2*a)

1. Lexical Analysis (Scanning)
Function: Breaks input into tokens.
Output: Tokens

Token Type	Lexeme
IDENTIFIER 1	R
= OPERATOR	=

(BRACKET	(
IDENTIFIER 2	b
* OPERATOR	*
IDENTIFIER 2	b
- OPERATOR	-
NUMBER	4.0
* OPERATOR	*
IDENTIFIER 3	a
* OPERATOR	*
IDENTIFIER 4	c
) BRACKET	)
/ OPERATOR	/
(BRACKET	(
NUMBER	2
* OPERATOR	*
IDENTIFIER 3	a
) BRACKET	)

2. Syntax Analysis (Parsing)
Function: Forms parse tree/AST following grammar.
Output: Abstract Syntax Tree (AST)

```
          =
        /   \
      ID1    /
            /   \
           -     *
          / \   / \
         *   *  2  ID3
        / \ / \
      ID2 ID2 *  ID4
             / \
           4.0 ID3
```

3. Semantic Analysis
Function: Type checking and validation.
Output: Symbol Table + Validated AST

```
          =
        /   \
      ID1    /
            /   \
           -     *
          / \   /   \
         *   *  intoreal(2)  ID3
        / \ / \
      ID2 ID2 *  ID4
             / \
           4.0 ID3
```

Identifier	Type
R	float
a	float
b	float
c	floak t

All operations involve numeric types → **Semantically valid**.

4. Intermediate Code Generation (ICG)
Function: Converts AST to 3-address code (TAC).
Output: TAC
t1 = ID2 * ID2
t2 = 4.0 * ID3
t3 = t2 * ID4
t4 = t1 - t3
t5=into real (2)
t6 = t5 * ID3
t7 = t4 / t6
ID1 = t7

5. Code Optimization
Function: Simplifies and improves TAC.
Optimized TAC:
t1 = ID2 * ID2
t2 = 4.0 * ID3
t3 = t2 * ID4
t4 = t1 - t3
t5 = 2.0 * ID3
ID1 = t4 / t5

6. Target Code Generation
Function: Produces assembly/machine code.
Output: Pseudo Assembly
MOV ID2, R1
MUL R1, R1
MOV ID3, R2
MUL #4.0, R2
MOV ID4, R3
MUL R2, R3
SUB R1, R3
MOV ID3, R4
MUL #2.0, R4
DIV R3, R4
MOV R4, ID1

5. I=p*n*r/100

1. Lexical Analysis (Scanning)
Function: Converts the input program into tokens.

Output: Tokens

Token Type	Lexeme
IDENTIFIER 1	I
= OPERATOR	=
IDENTIFIER 2	p
* OPERATOR	*
IDENTIFIER 3	n
* OPERATOR	*
IDENTIFIER 4	r
/ OPERATOR	/
NUMBER	100

2. Syntax Analysis (Parsing)
Function: Checks grammar and produces a parse tree or AST.
Output: Abstract Syntax Tree (AST)

```
      =
     / \
  Id1   /
       / \
      *   100
     / \
    *  Id4
   / \
Id2 Id3
```

3. Semantic Analysis
Function: Verifies meaning—type checking, declaration checking, etc.
Output: Symbol Table + Validated AST

Identifier	Type
I	Int
p	Int
n	Int
r	int

4. Intermediate Code Generation (ICG)
Function: Converts AST to **3-address code**.
Output: 3-Address Code (TAC)
t1 = ID2 * ID3
t2 = t1 * ID4
t3 = t2 / 100
ID1 = t3

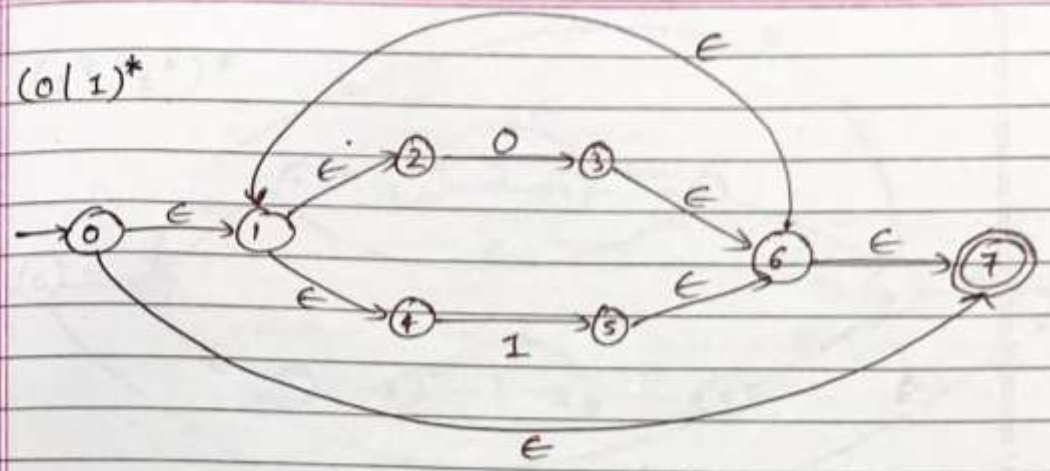
5. Code Optimization
Function: Makes TAC more efficient.
Optimized TAC

t1 = ID2 * ID3
t2 = t1 * ID4
ID1 = t2 / 100

6. Target Code Generation
Function: Generates assembly-like machine instructions.
Output: Pseudo Assembly
MOV ID2, R1
MOV ID3, R2
MUL R1, R2
MOV ID4, R3
MUL R2, R3
DIV #100, R3
MOV R3, ID1

Lab - 5

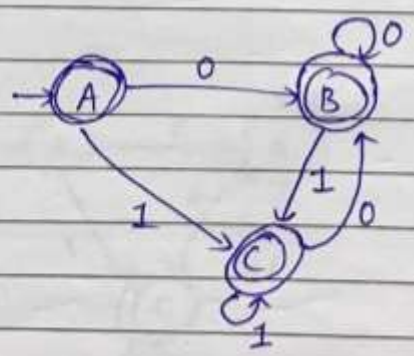
1. $(0|1)^*$



—: NFA - ε :—

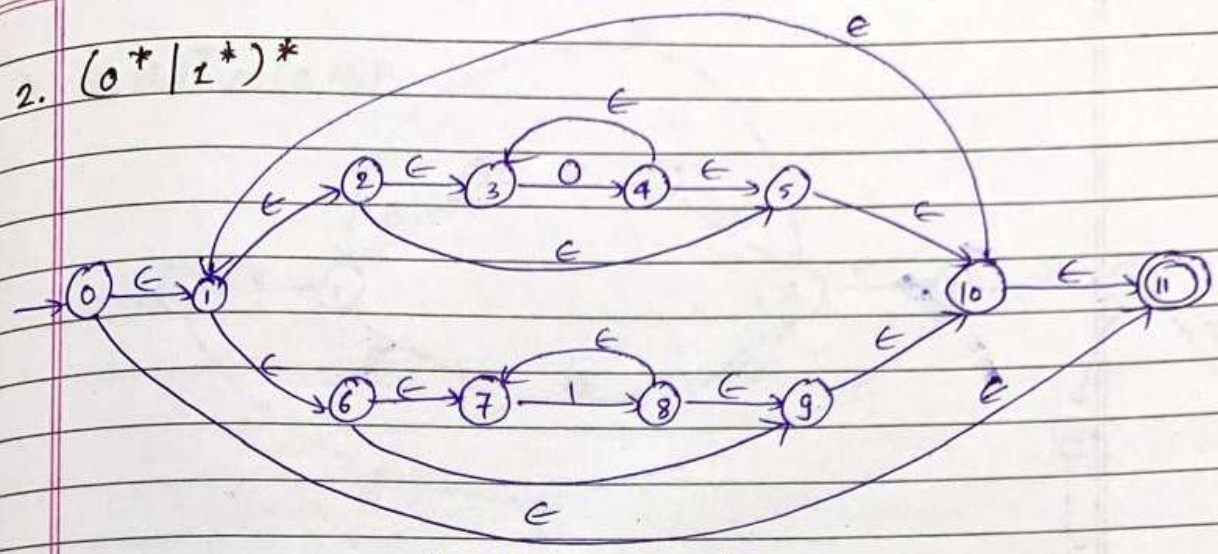
NFA-ε to DFA:

	0	1
→*A = {0, 1, 2, 4, 7}	B	C
*B = {3, 6, 1, 2, 4, 7}	B	C
*C = {5, 6, 1, 2, 4, 7}	B	C



—: DFA :—

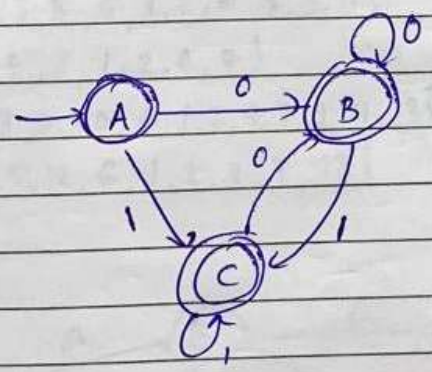
2. $(0^* | 1^*)^*$



—! NFA - ε !—

NFA -ε to DFA:

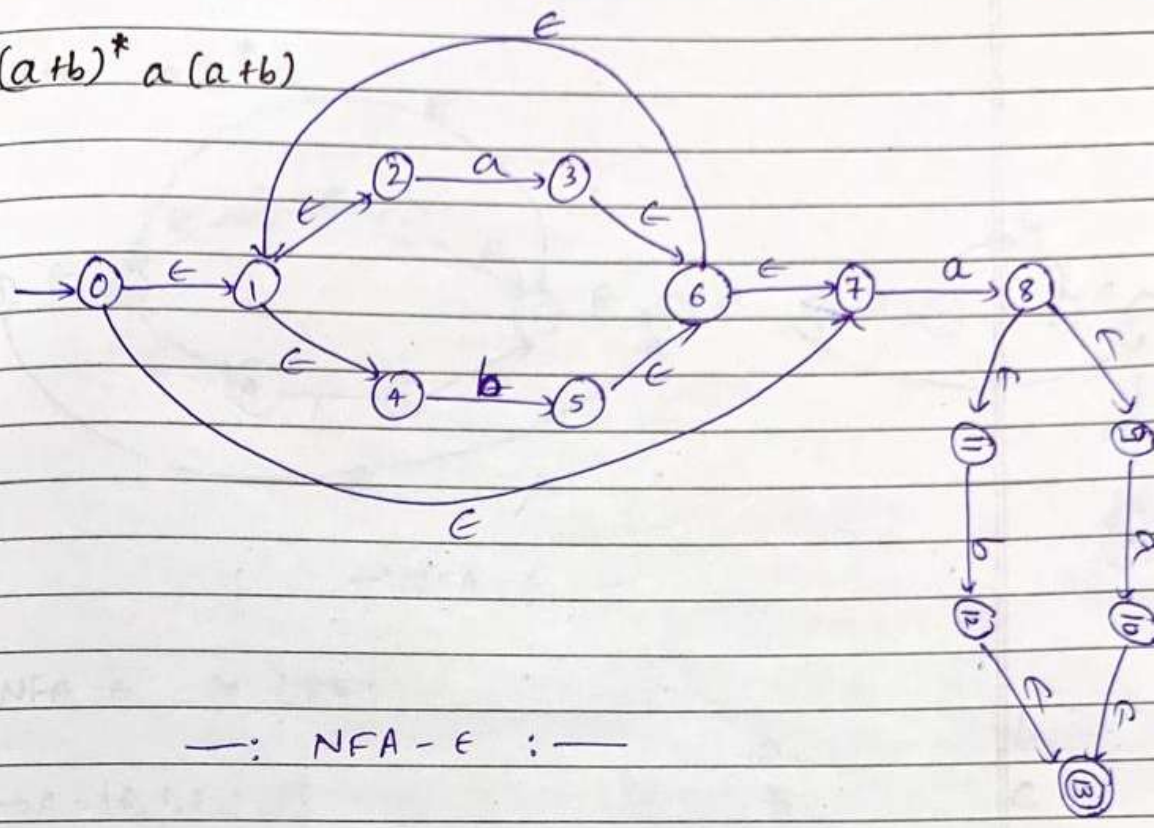
	0	1
→* A = {0, 1, 2, 3, 5, 10, 11, 6, 7, 9}	B	C
*B = {4, 3, 5, 10, 1, 2, 3, 6, 7, 9, 11}	B	C
*C = {8, 7, 9, 10, 1, 2, 3, 5, 6, 11}	B	C



—! DFA !—

Lab – 5 question 3 is given in ppt.

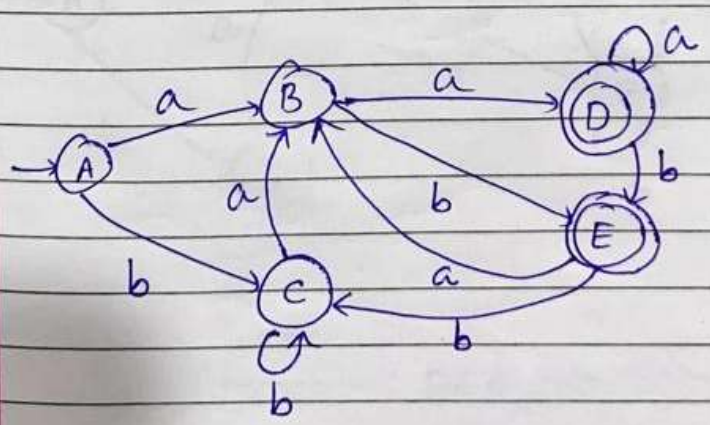
4. $(a+b)^* a (a+b)$



—: NFA - ε :—

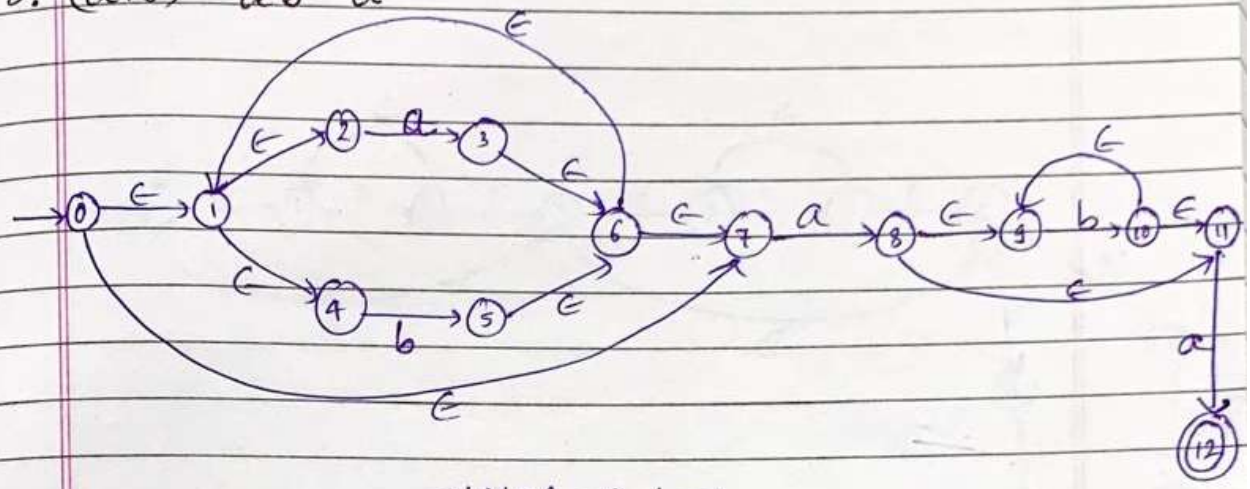
NFA - ε to DFA

	a	b
→ A = {0, 1, 2, 4, 7}	B	C
B = {3, 8, 6, 1, 2, 4, 7, 9, 11}	D	E
C = {5, 6, 1, 2, 4, 7}	B	C
* D = {3, 8, 10, 6, 1, 2, 4, 7, 9, 11, 13}	D	E
* E = {5, 12, 6, 1, 2, 4, 7, 13}	B	C



—: DFA :—

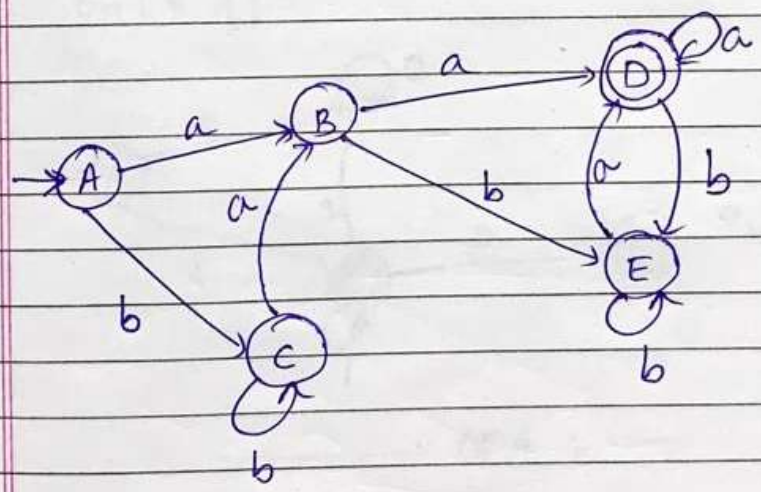
5. $(a+b)^* ab^* a$



—: NFA - ε :—

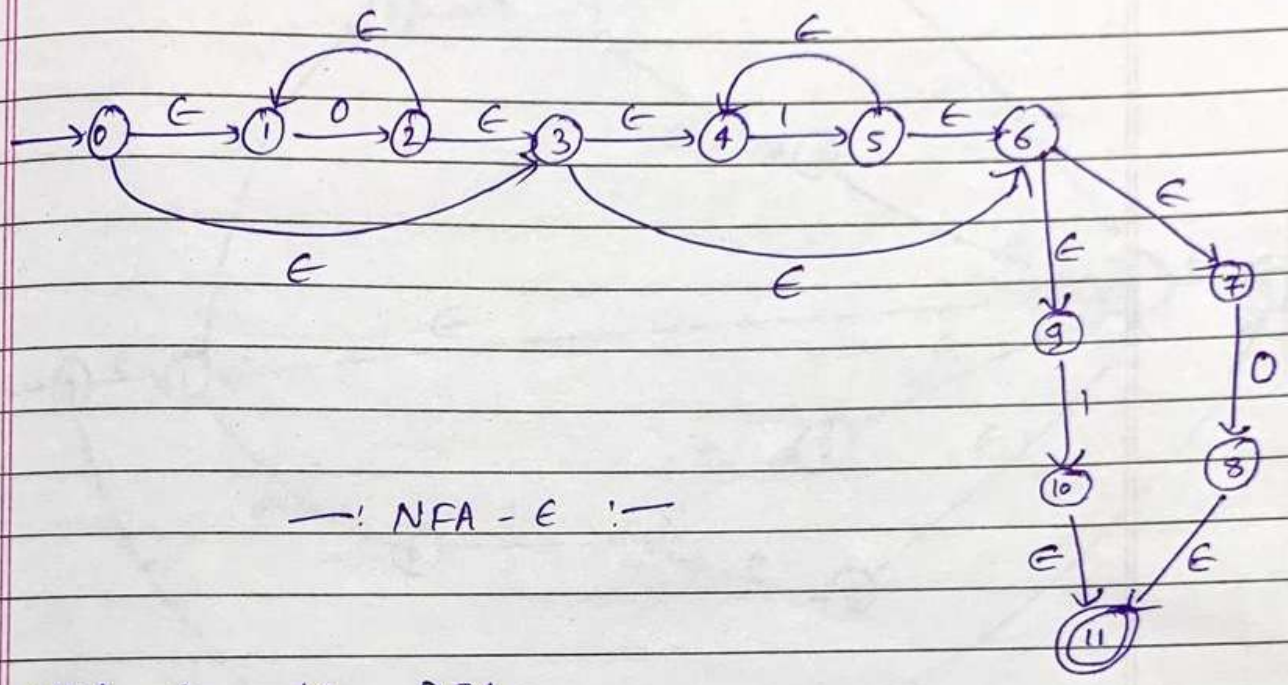
NFA - ε to DFA:

	a	b
→ A = {0, 1, 2, 4, 7}	B	C
B = {3, 8, 6, 1, 2, 4, 7, 9, 11}	D	E
C = {5, 6, 1, 2, 4, 7}	B	C
* D = {3, 8, 12, 6, 1, 2, 4, 7, 9, 11}	D	E
E = {5, 10, 6, 1, 2, 4, 7, 11, 9}	D	E



——— : DFA : ———

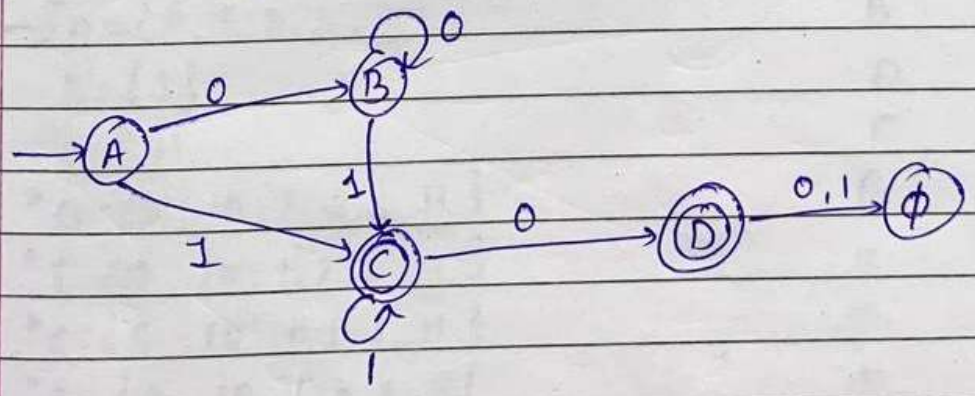
6. $0^* 1^* (0|1)$ #



—: NFA - ϵ :—

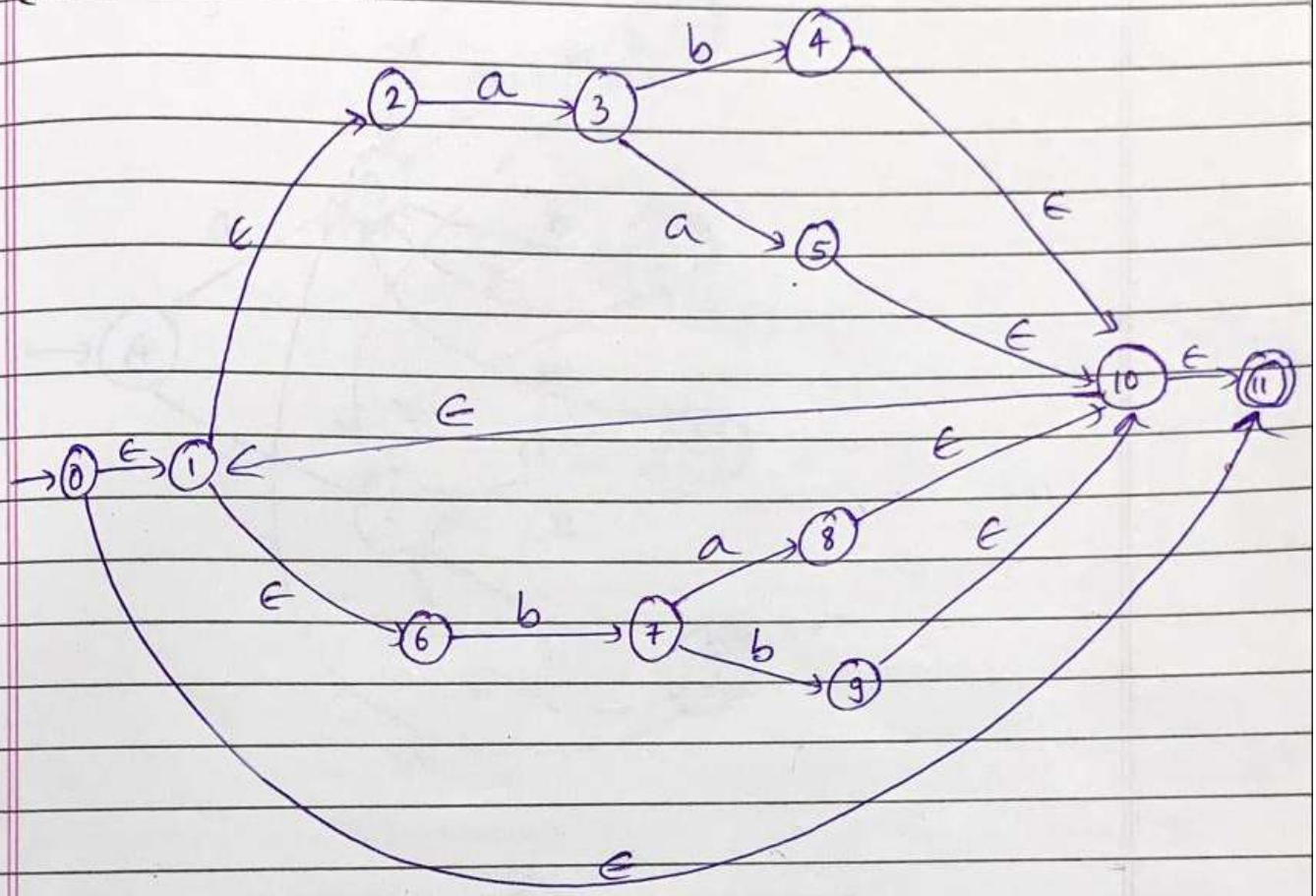
NFA -ϵ to DFA

	0	1
→ A = {0, 1, 3, 4, 6, 7, 9}	B	C
B = {2, 8, 1, 3, 4, 6, 7, 9}	B	C
*C = {5, 10, 4, 6, 7, 9, 11}	D	C
*D = {8, 11}	ϕ	ϕ



—: DFA :—

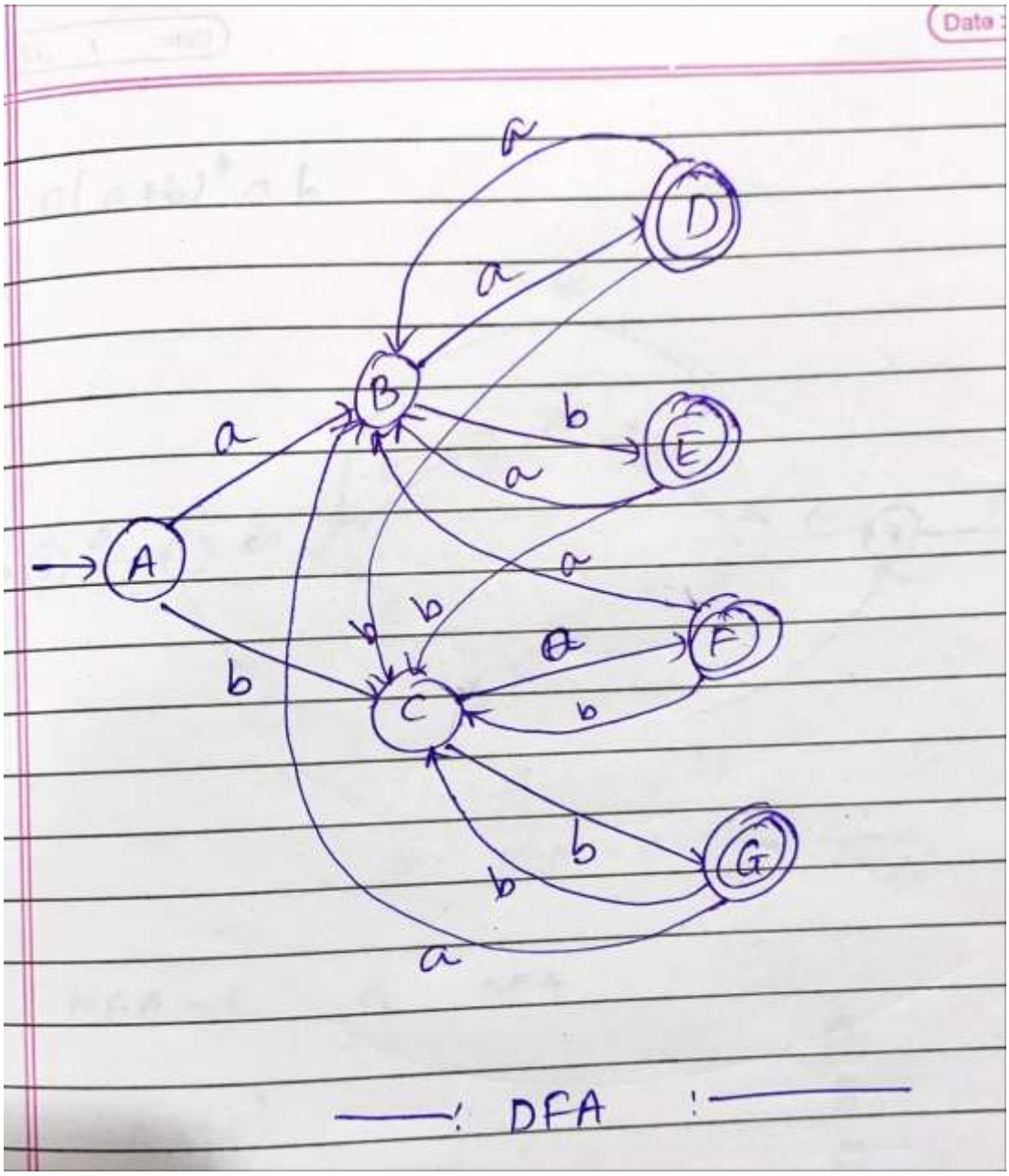
7. $(ab|aa|ba|bb)^*$



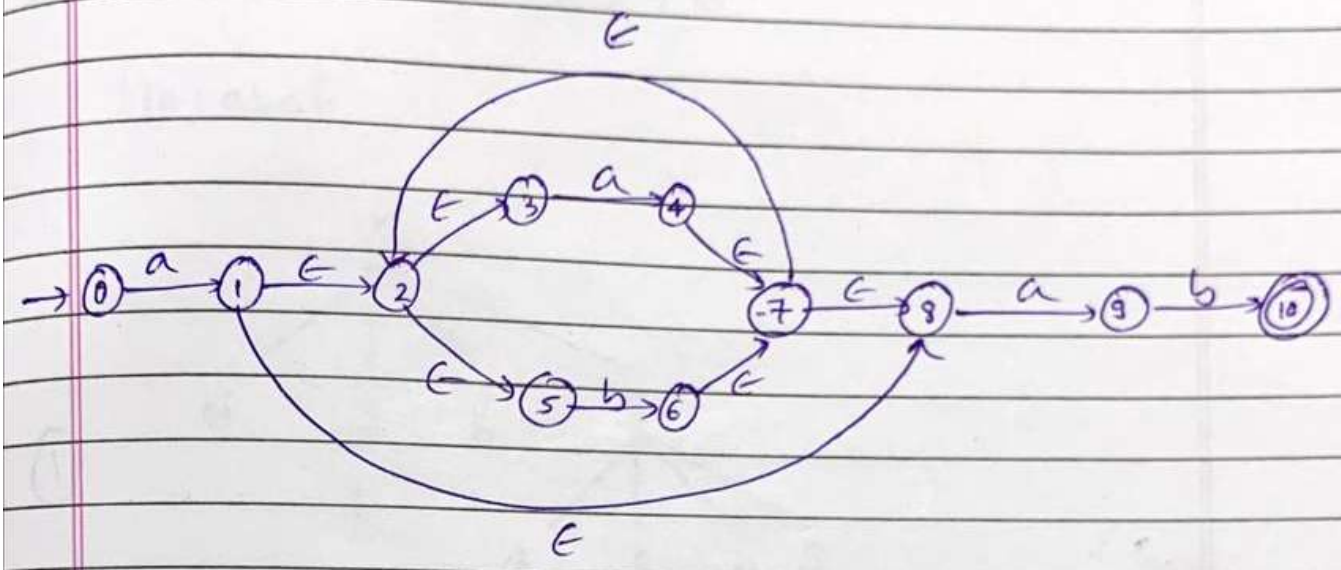
—: NFA – ε :—

NFA – ε to DFA:

	a	b
→*A = {0, 1, 2, 6, 11}	B	C
B = {3}	D	E
C = {7}	F	G
*D = {5, 10, 1, 2, 6, 11}	B	C
*E = {4, 10, 1, 2, 6, 11}	B	C
*F = {8, 10, 1, 2, 6, 11}	B	C
*G = {9, 10, 1, 2, 6, 11}	B	C


A
B
C
D
E
F
G
a
b
DFA

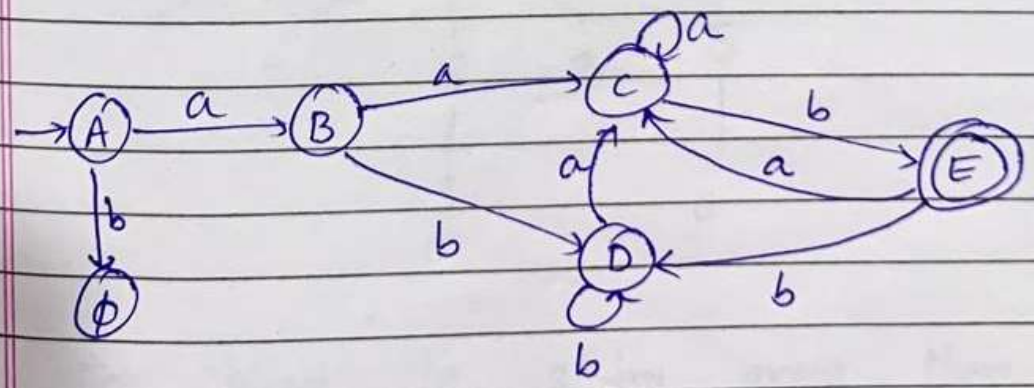
8. $a(a+b)^* ab$



—: NFA – ε :—

NFA – ε to DFA

	a	b
→ A = {0}	B	ϕ
B = {1, 2, 3, 5, 8}	C	D
C = {4, 9, 7, 2, 3, 5, 8}	C	E
D = {6, 7, 2, 3, 5, 8}	C	D
* E = {6, 10, 7, 2, 3, 5, 8}	C	D



—: DFA :—

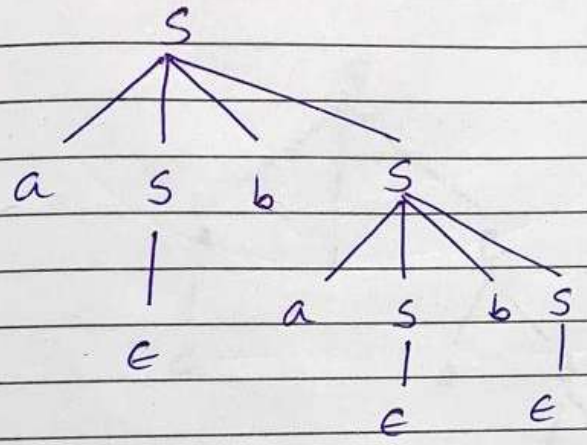
Lab-6

Check whether grammar is ambiguous or not?

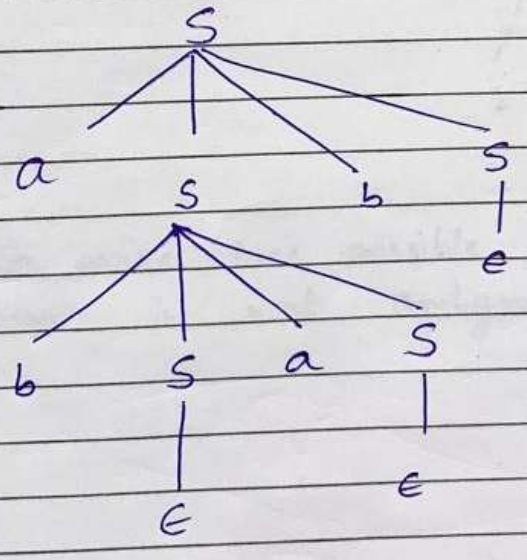
1. $S \rightarrow aSbS \mid bSaS \mid \epsilon$

i/p : abab

(1)



(2)



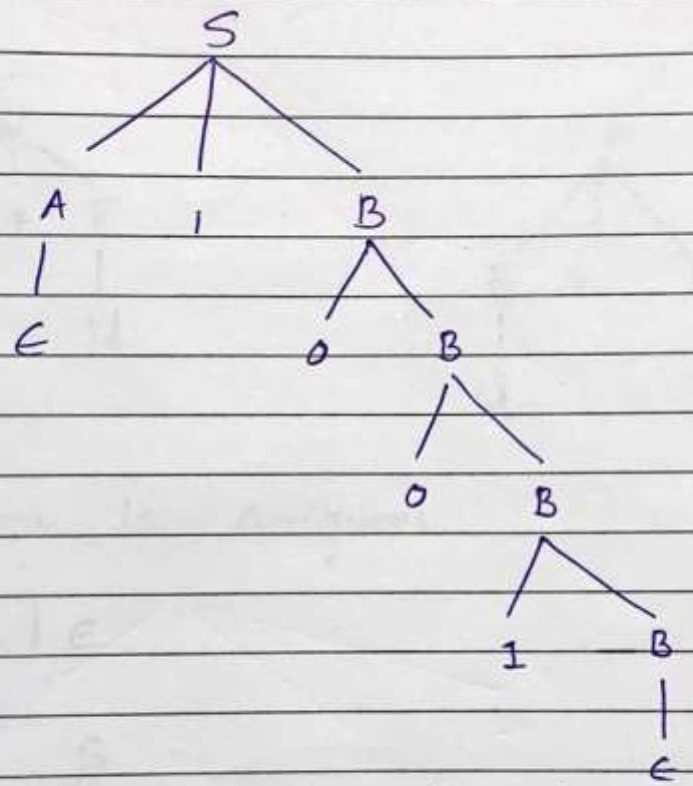
For given i/p string more than one parse trees are ~~three~~ possible as per figure

∴ Grammar is ambiguous.

Page No.
Date : / /20

2. $S \rightarrow A1B$
$A \rightarrow 0A \mid \epsilon$
$B \rightarrow 0B \mid 1B \mid \epsilon$

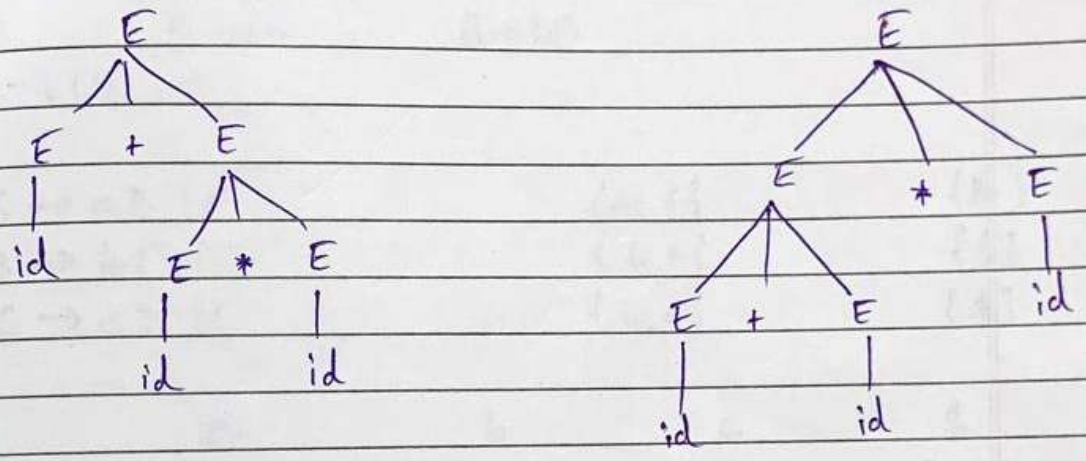
i/p: 1001



No other parse tree possible for same i/p.
∴ Grammar is not ambiguous.

3. $E \rightarrow E+E \mid E*E \mid id \mid (E) \mid -E$

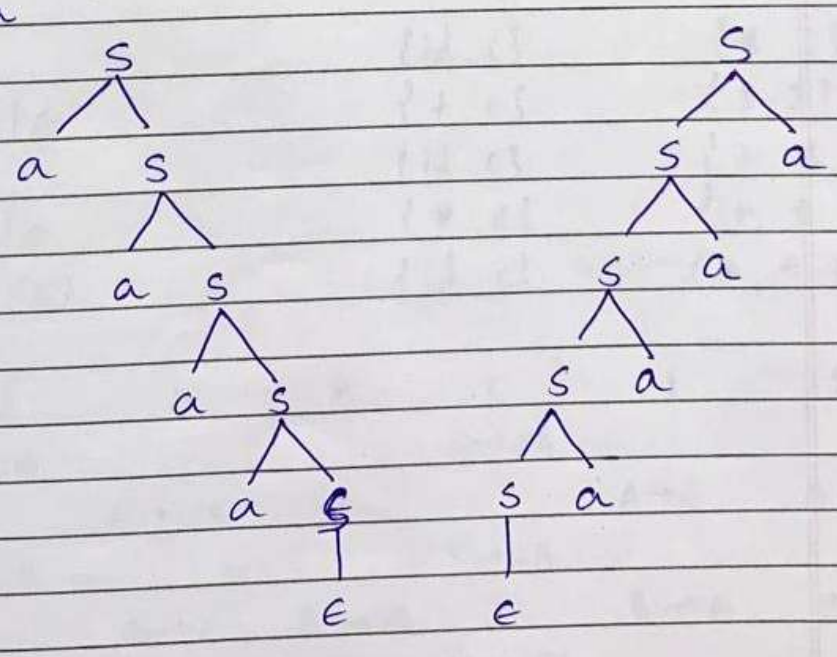
i/p: id + id * id



∴ Grammar is ambiguous.

4. $S \rightarrow aS \mid Sa \mid \epsilon$

i/p: aaaa



∴ Grammar is ambiguous.

7. **Perform Left Recursion and Left Factoring Examples**

Eliminate Left recursion and perform left factoring on given grammar.

Left recursion

1. A→Abd | Aa | a
   B→Be | b

A→ aA'
A'→ bdA' | aA' | ϵ
B→ bB'
B'→ eB' | ϵ

2. A→AB | AC | a | b
   A→ aA' | bA'
   A'→ BA' | CA' | ϵ

3. S→A | B
   A→ABC | Acd | a | aa
   B→Bee | b

S→A|B
A→ aA' | aaA'
A'→ BCA' |cdA' | ϵ
B → bB'
B'→ eeB' | ϵ

4. S→(L) | a
   L→L , S | S

S→ (L)|a
L→ SL'
L'→ ,SL' | ϵ

Left factoring

1. S→iEtS | iEtSeS | a
   S→ iEtSS' | a
   S'→ ϵ | eS

2. A→ ad | a | ab | abc | x
   A→ aA'|x
   A' → d | ϵ | b | bc
   Finally:
   A'→ d | ϵ | bB'
   B' → ϵ | c

3. S→A
   A→Ad | Ae | aB | aC
   B→bBC | f
   C→g
   S→ A
   A→ aBA' | aCA'
   A'→ dA' | eA' | ϵ
   B→ bBC | f
   C → g
   Finaly:
   S→ A
   A→ aA''
   A''→ BA' | CA'
   A'→ dA' | eA' | ϵ
   B→ bBC | f
   C → g

Lab - 8

Page No.
Date : / /20

LL(1)

	First	Follow
1. $S \to aBa$	$\{a\}$	$\{\$\}$
$B \to bB \mid \epsilon$	$\{b, \epsilon\}$	$\{a\}$

	a	b	$
S	$S \to aBa$		
B	$B \to \epsilon$	$B \to bB$	

$\therefore$ LL(1) ✓

2. $S \to aB \mid \epsilon$	$\{a, \epsilon\}$	$\{\$\}$
$B \to bC \mid \epsilon$	$\{b, \epsilon\}$	$\{\$\}$
$C \to cS \mid \epsilon$	$\{c, \epsilon\}$	$\{\$\}$

	a	b	c	$
S	$S \to aB$			$S \to \epsilon$
B		$B \to bC$		$B \to \epsilon$
C			$C \to cS$	$C \to \epsilon$

$\therefore$ LL(1) ✓

3. $E \to TA$	$\{id, (\}$	$\{\$,)\}$
$A \to +TA \mid \epsilon$	$\{+, \epsilon\}$	$\{\$,)\}$
$T \to VB$	$\{id, (\}$	$\{+, \$,)\}$
$B \to *VB \mid \epsilon$	$\{*, \epsilon\}$	$\{+, \$,)\}$
$V \to id \mid (E)$	$\{id, (\}$	$\{*, +, \$,)\}$

	id	+	*	(	)	$
E	$E \to TA$			$E \to TA$		
A		$A \to +TA$			$A \to \epsilon$	$A \to \epsilon$
T	$T \to VB$			$T \to VB$		
B		$B \to \epsilon$	$B \to *VB$		$B \to \epsilon$	$B \to \epsilon$
V	$V \to id$			$V \to (E)$		

$\therefore$ LL(1) ✓

Stack	i/p	o/p
E $	id + id $	—
TA $	id + id $	E → TA
VBA $	id + id $	T → VB
id BA $	id + id $	V → id
BA $	+ id $	—
A $	+ id $	B → ∈
+ TA $	+ id $	A → + TA
TA $	id $	—
VB A $	id $	T → VB
id BA $	id $	V → id
BA $	$	—
A $	$	B → ∈
A $	$	A → ∈

∴ String is valid.

so (id + id) + (id + id) is valid string, parse the same way.

4.

	First	Follow
E → BA	{ true, false }	{ $ }
A → & BA | ∈	{ &, ∈ }	{ $ }
B → true | false	{ true, false }	{ &, $ }

	true	false	&	$
E	E → BA	E → BA		
A			A → & BA	A → ∈
B	B → true	B → false		

∴ LL(1) ✓

	First	Follow
5. $S \to AaAb \mid BbBa$	$\{a, b\}$	$\{\$\}$
$A \to \epsilon$	$\{\epsilon\}$	$\{a, b\}$
$B \to \epsilon$	$\{\epsilon\}$	$\{a, b\}$

	a	b	$
S	$S \to AaAb$	$S \to BbBa$	
A	$A \to \epsilon$	$A \to \epsilon$	
B	$B \to \epsilon$	$B \to \epsilon$	

$\therefore$ LL(1) ✓

6. $S \to aAB \mid bA \mid \epsilon$	$\{a, b, \epsilon\}$	$\{\$\}$
$A \to aAb \mid \epsilon$	$\{a, \epsilon\}$	$\{b, \$\}$
$B \to bB \mid \epsilon$	$\{b, \epsilon\}$	$\{\$\}$

	a	b	$
S	$S \to aAB$	$S \to bA$	$S \to \epsilon$
A	$A \to aAb$	$A \to \epsilon$	$A \to \epsilon$
B		$B \to bB$	$B \to \epsilon$

$\therefore$ LL(1) ✓

7. $S \to iCtSA \mid a$	$\{i, a\}$	$\{\$, e\}$
$A \to eS \mid \epsilon$	$\{e, \epsilon\}$	$\{\$, e\}$
$C \to b$	$\{b\}$	$\{t\}$

	i	b	e	a	t	$
S	$S \to iCtSA$			$S \to a$		
A			$A \to eS$, $A \to \epsilon$			$A \to \epsilon$
C		$C \to b$				

$\therefore$ LL(1) ✗ (Not LL(1))

9. Recursive Decent Parsing

E→E + T | T

T→T * F | F

F→id

Page No.
Date : / /20

Lab – 9

RDP:

9. $E \rightarrow E+T \mid T$
$T \rightarrow T*F \mid F$
$F \rightarrow id$

Remove left Recursion

$E \rightarrow TE'$
$E' \rightarrow +TE' \mid \epsilon$
$T \rightarrow FT'$
$T' \rightarrow *FT' \mid \epsilon$
$F \rightarrow id.$

RDP : Procedures:

```
E()
{   T();
    E'();
}

T()
{   F();
    T'();
}

E'()
{   if (lookahead == '+')
    {   match('+');
        T();
        E'();
    }
    else
        return;
}

T'()
{   if (lookahead == '*');
    {   match('*');
        F();
        T'();
    }
    else
        return;
}
```

```
F()
{   if (lookahead == 'id')
            match('id);
    elser
            error()
}

match(char  token)
{
    if ( lookahead == token)
            lookahead = next token;
    else
            error()
}
```

10. Check the following grammar is operator grammar or not. Justify your answer. Also prepare operator precedence matrix and graph.

1. **E→E + T | T**

   T→T * F | F

   F→(E) | id

   See: next page

2. **E→EAE | id**

   A→+ | *

   Two non terminals are in adjacent, so grammar is not operator grammar

3. **E→EOE**

   E→id

   O→* | + | -

   Two non terminals are in adjacent, so grammar is not operator grammar

Lab-10

Date : / /20

1. $E \rightarrow E+T \mid T$
$T \rightarrow T*F \mid F$
$F \rightarrow (E) \mid id$

given grammar is operator grammar

	Leading	Trailing
E	{ (, id, *, + }	{), id, *, + }
T	{ (, id, * }	{), id, * }
F	{ (, id }	{), id }

→ Operator precedence matrix

	+	*	(	)	id	$
+	>	<·	<·	>	<·	>
*	>	>	<·	>	<·	>
(	<·	<·	<·	≐	<·	
)	>	>		>		>
id	>	>		>		>
$	<·	<·	<·		<·	

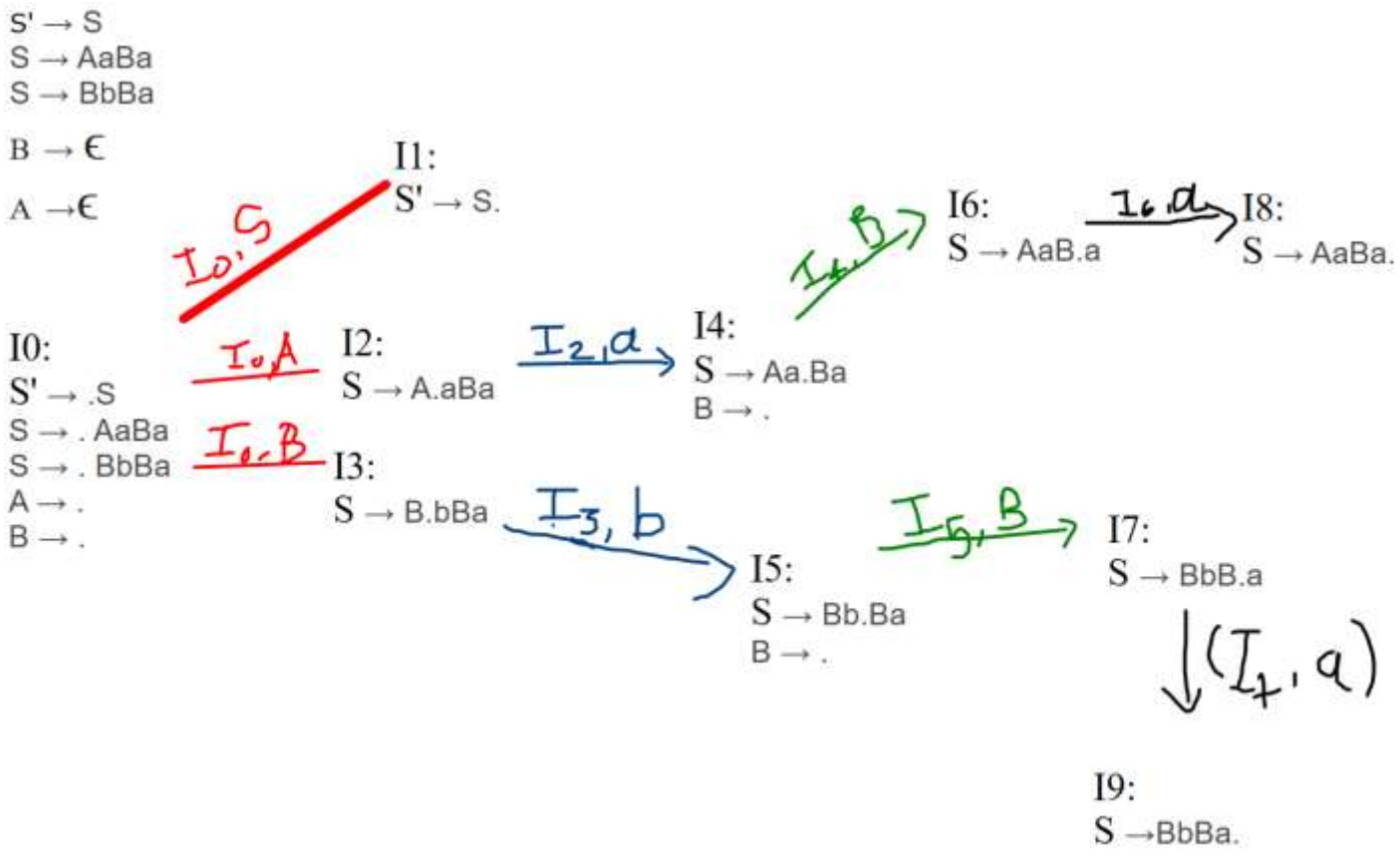
1. $aAb \Rightarrow a \doteq b \quad \therefore (\doteq)$
2. $aA \Rightarrow a <\cdot \text{Leading}(A) \quad \therefore \quad + <\cdot \text{Leading}(T)$
$* <\cdot \text{Leading}(F)$
$(<\cdot \text{Leading}(E)$
3. $Ab \Rightarrow \text{Trailing}(A) \cdot> b \quad \therefore \quad \text{Trailing}(E) \cdot> +$
$\text{Trailing}(T) \cdot> *$
$\text{Trailing}(E) \cdot>)$
4. $\$ <\cdot \text{Leading}(E)$
5. $\text{Trailing}(E) \cdot> \$$

12. Construct SLR parsing table for given grammar.

1. S→AaBa | BbBa

   B→Є

   A→Є



1. SLR Parsing Table

	action			goto		
	a	b	$	S	B	A
0	R4/R3	R3		1	3	2
1			Accept			
2	S4					
3		S5				
4	R3	R3			6	
5	R3	R3			7	
6	S8					
7	S9					
8			R1			
9			R2			

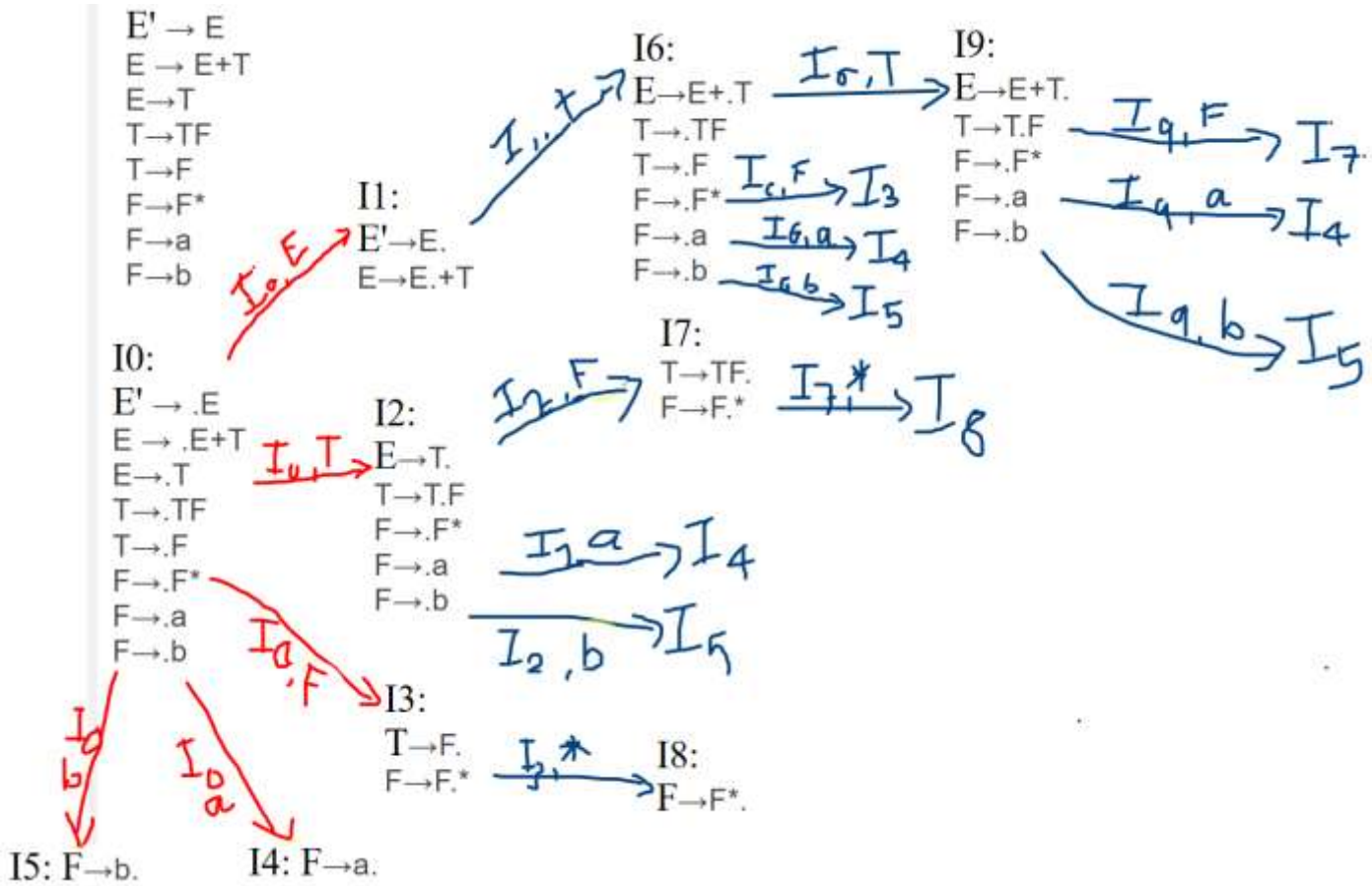
		Follow
1)	$S \rightarrow AaBa$	$\{\$\}$
2)	$S \rightarrow BbBa$	
3)	$B \rightarrow \epsilon$	$\{a, b\}$
4)	$A \rightarrow \epsilon$	$\{a\}$

∴ There is reduce - reduce conflict.
SLR Parsing is not possible

2. E→ E+T | T

   T→ TF | F

   F→ F* | a | b



2. 1) $E \rightarrow E+T$
2) $E \rightarrow T$
3) $T \rightarrow TF$
4) $T \rightarrow F$
5) $F \rightarrow F*$
6) $F \rightarrow a$
7) $F \rightarrow b$

	Follow
E	{$, +}
T	{a, b, $, +}
F	{*, a, b, $, +}

SLR Parsing Table

	+	*	a	b	$	E	T	F
0			S4	S5		1	2	3
1	S6				accept			
2	R2		S4	S5	R2			7
3	R4	S8	R4	R4	R4			
4	R6	R6	R6	R6	R6			
5	R7	R7	R7	R7	R7			
6			S4	S5			9	3
7	R3	S8	R3	R3	R3			
8	R5	R5	R5	R5	R5			
9	R1		S4	S5	R1			7

Construct CLR parsing table for given grammar.

1. **S→CC**

   C→cC | d

   Canonical LR(1) Item set:

   I0

   S→.S, $

   S→.CC,$

   C→.cC,c/d

   C→.d,c/d

   I1: GOTO(I0,S)

   S'→S., $

   I2: GOTO (I0,C)

   S→C.C,$

   C→.cC,$

   C→.d,$

   I3:GOTO(I0,c)

   C→c.C,c/d

   C→.cC,c/d →I3

   C→.d,c/d →I4

   I4: GOTO(I0,d)

   C→d. , c/d

   I5: GOTO(I2,C)

   S→CC., $

   I6:GOTO (I2,c)

   C→c.C,$

   C→.cC,$ →I6

   C→.d,$ →I7

   I7: GOTO (I2, d)

C→d.,$

I8:GOTO(I3,C)

C→cC. , c/d

I9:GOTO(I6,C)

C→cC. , $

Canonical Parsing Table:

State	action			goto	
	c	d	$	S	C
0	S3	S4		1	2
1			Acc		
2	S6	S7			5
3	S3	S4			8
4	r3	r3			
5			r1		
6	S6	S7			9
7			r3		
8	r2	r2			
9			r2		

S→CC __ (1)

C→cC __ (2)

C→d __ (3)

2. S→aSA | Є

A→bS |c

I0
S'→.S,$
S→.aSA,$
S→. , $

I1: GOTO(I0,S)
S'→S. , $

I2:GOTO(I0,a)
S→a.SA,$
S→.aSA, b/c
S→. , b/c

I3: GOTO(I2,S)
S→aS.A,$
A→.bS,$
A→.c,$

I4: GOTO(I2,a)
S→a.SA, b/c
S→.aSA, b/c →I4
S→. , b/c

I5:GOTO(I3,A)
S→aSA. , $

I6: GOTO(I3,b)
A→b.S,$
S→.aSA, $ →I2
S→. , $

I7: GOTO(I3,c)
A→c. , $

I8: GOTO (I4,S)
S→aS.A , b/c
A→.bS , b/c
A→.c , b/c

I9: GOTO (I6,S)
A→bS. , $

I10:GOTO (I8, A)
S→aSA. , b/c

I11:GOTO(I8,b)
A→b.S , b/c
S→.aSA , b/c
S→. , b/c

I12:GOTO(I8, c)
A→c. , b/c

I13: GOTO(I11,S)
A→bS. , b/c

4. 1) $S \rightarrow aSA$
2) $S \rightarrow \epsilon$
3) $A \rightarrow bS$
4) $A \rightarrow c$

CLR Parsing Table

	Action				goto	
	a	b	c	$	S	A
0	S2			R2	1	
1				Accept		
2	S4	R2	R2		3	
3		S6	S7			5
4	S4	R2	R2		8	
5				R1		
6	S2			R2	9	
7				R4		
8		S11	S12			10
9				R3		
10		R1	R1			
11		R2	R2		13	
12		R4	R4			
13		R3	R3			

Construct LALR parsing table for given grammar.

1. **S→CC**

   C→cC | d

   As per question 1 of CLR, create canonical LR(1) item set and merge item set 36, 47, and 89

LALR Parsing Table

	Action			goto	
state	c	d	$	S	C
0	S36	S47		1	2
1			Acc		
2	S36	S47			5
36	S36	S47			89
47	r3	r3	r3		
5			r1		
89	r2	r2	r2		

6. LALR: $S' \to S$

1) $S' \to Aa$
2) $S \to bAc$
3) $S \to dc$
4) $S \to bda$
5) $A \to d$

I_0:
$S' \to \cdot S, \$$
$S \to \cdot Aa, \$$
$S \to \cdot bAc, \$$
$S \to \cdot dc, \$$
$S \to \cdot bda, \$$
$A \to \cdot d, a$

I_1: goto(I_0, S)
$S' \to S \cdot, \$$

I_2: goto(I_0, A)
$S \to A \cdot a, \$$

I_3: goto(I_0, b)
$S \to b \cdot Ac, \$$
$S \to b \cdot da, \$$
$A \to \cdot d, c$

I_4: goto(I_0, d)
$S \to d \cdot c, \$$
$A \to d \cdot, a$

I_5: goto(I_2, a)
$S \to Aa \cdot, \$$

I_6: goto(I_3, A)
$S \to bA \cdot c, \$$

I_7: goto(I_3, d)
$S \to bd \cdot a, \$$
$A \to d \cdot, c$

I_8: goto(I_4, c)
$S \to dc \cdot, \$$

I_9: goto(I_6, c)
$S \to bAc \cdot, \$$

I_{10}: goto(I_7, a)
$S \to bda \cdot, \$$

No items are same
∴ None can merge

LALR Parsing Table

	Action					goto	
	a	b	c	d	$	S	A
0		S3		S4		1	2
1					Accept		
2	S5						
3				S7			6
4	R5		S8				
5					R1		
6			S9				
7	S10		R5				
8					R3		
9					R2		
10					R4		

13. Write SDT to convert infix to postfix notation and draw annotated parse tree for a given string.

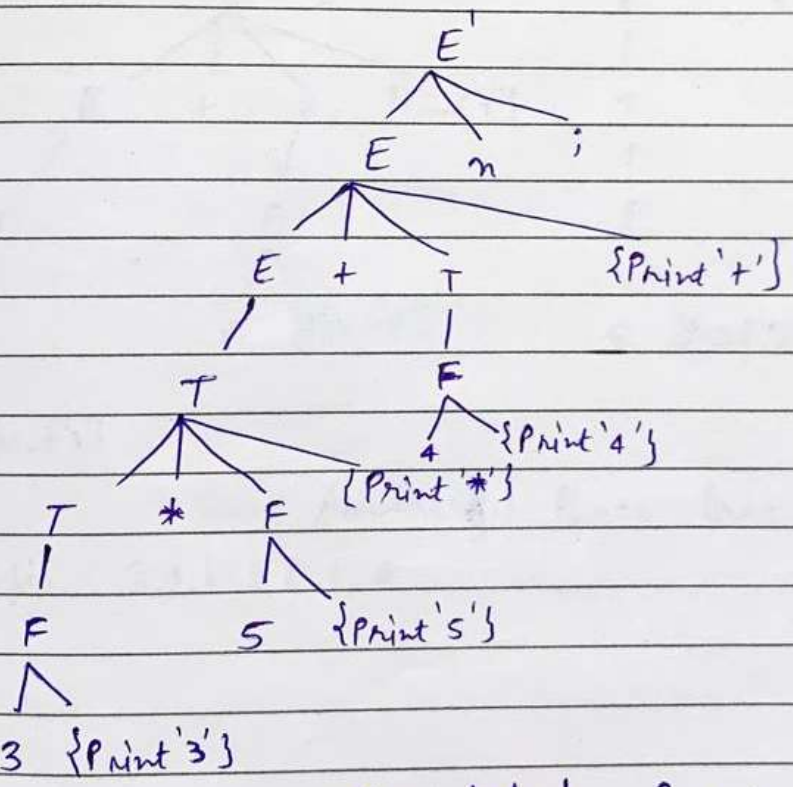
Lab-13

Page No.
Date : / /20

SDT to convert infix to postfix

SDT:
$E' \rightarrow En;$
$E \rightarrow E + T$ {print '+'}
$E \rightarrow T$
$T \rightarrow T * F$ {print '*'}
$T \rightarrow F$
$F \rightarrow NUM$ {Print NUMval}
$F \rightarrow (E)$

1. 3 * 5 + 4n;

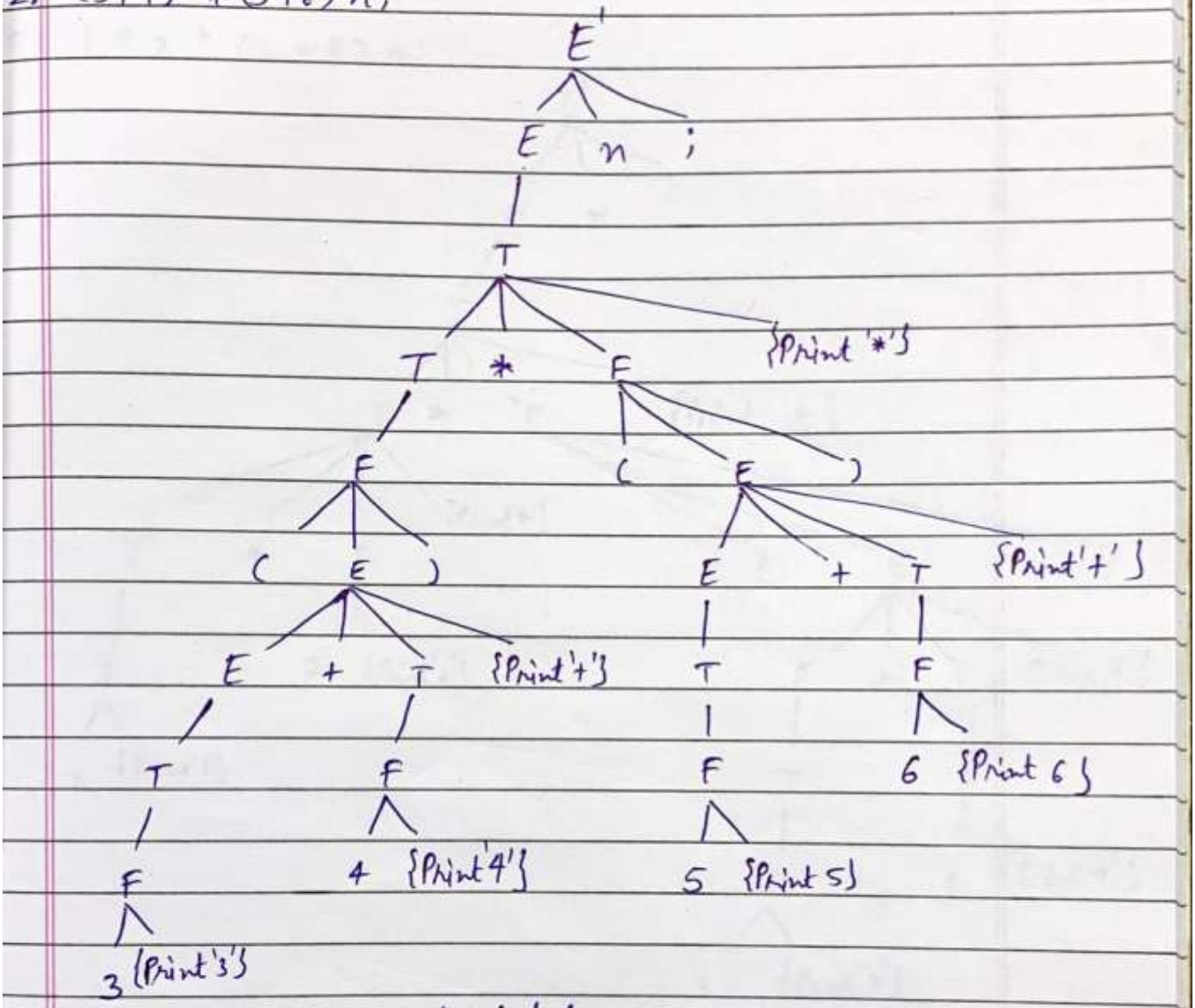


—: Annotated Parse tree :—

Postfix : 3 5 * 4 +

Page No.
Date : / /20

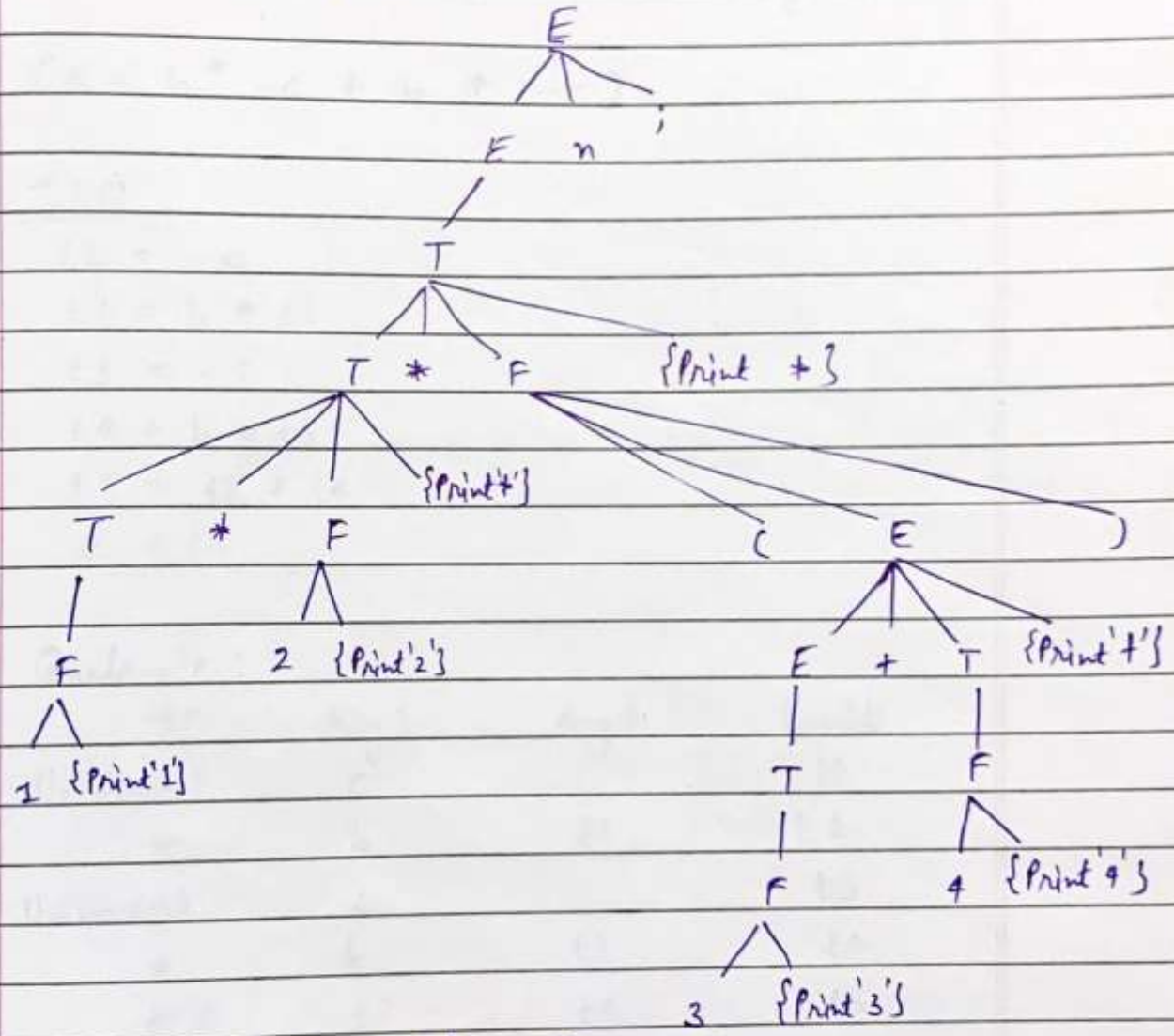
2. (3+4) * (5+6) n;



—! Annotated Parse tree !—

Postfix : 3 4 + 5 6 + *

3. 1 * 2 * (3 + 4) n;



—: Annotated Parse Tree: —

Postfix: 12*34+*

Lab – 14

Page No.
Date : / /20

TAC: Three Address Code generation

1. (a = b * -c + b * -c)

TAC:

```
t1 = -c
t2 = b * t1
t3 = -c
t4 = b * t3
t5 = t2 + t4
a = t5
```

Quadruple:

OP	Arg 1	Arg 2	Result
Uniminus(-)	c		t1
*	b	t1	t2
Uniminus(-)	c		t3
*	b	t3	t4
+	t2	t4	t5
=	t5		a

Triple:

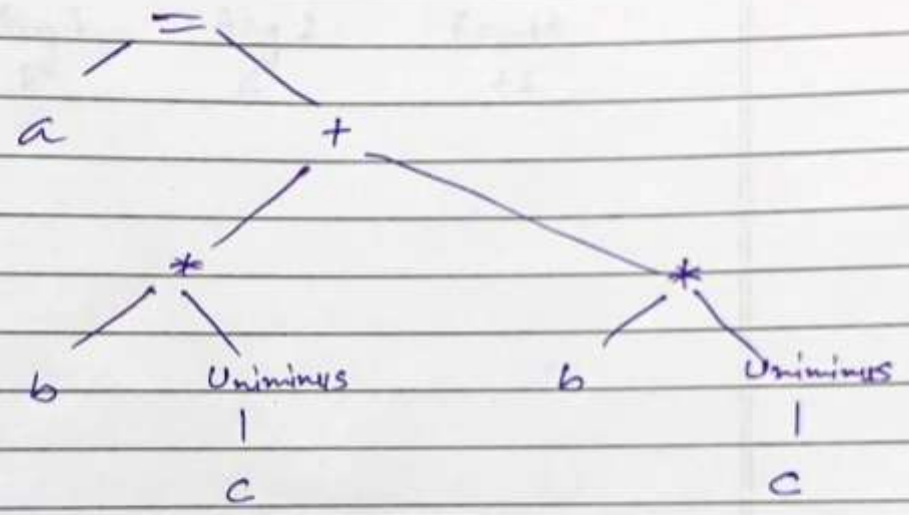
	OP	Arg 1	Arg 2
(0)	Uniminus	c	
(1)	*	b	(0)
(2)	Uniminus	c	
(3)	*	b	(2)
(4)	+	(1)	(3)
(5)	=	a	(4)

Indirect Triple:

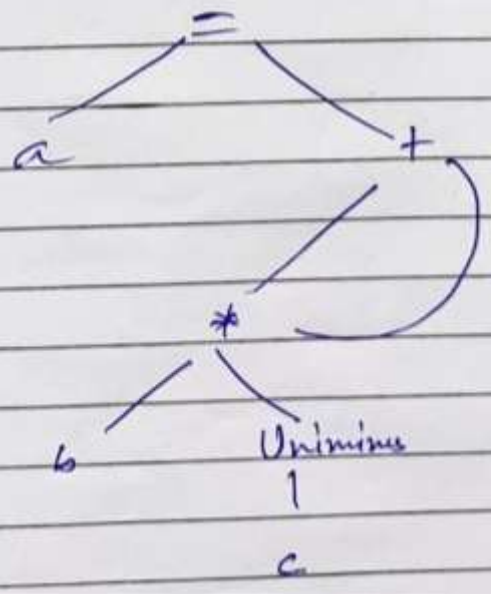
(0) (14)
(1) (15)
(2) (16)
(3) (17)
(4) (18)
(5) (19)

Here is given refercce of triple - 14, 15, 16...

AST : Abstract syntax tree



DAG: Directed Acyclic Graph



2. a * -(b+c)

TAC:

t1 = b+c

t2 = Uniminus (~~b+c~~ t1)

t3 = a * t2

Quadruple:

OP	Arg 1	Arg 2	Result
+	b	c	t1
Uniminus	t1		t2
*	a	t2	t3

Triple:

	OP	Arg 1	Arg 2
(0)	+	b	c
(1)	Uniminus	(0)	—
(2)	*	a	(1)

Indirect triple

(0)	(17)
(1)	(18)
(2)	(19)

Page No.
Date : / /20

3. a + a*(b-c) + (b-c)*d

TAC:

t1 = b-c
t2 = a*t1
t3 = a+t2
t4 = b-c
t5 = t4*d
t6 = t3+t5

Quadruple

OP	Arg 1	Arg 2	Result
-	b	c	t1
*	a	t1	t2
+	a	t2	t3
-	b	c	t4
*	t4	d	t5
+	t3	t5	t6

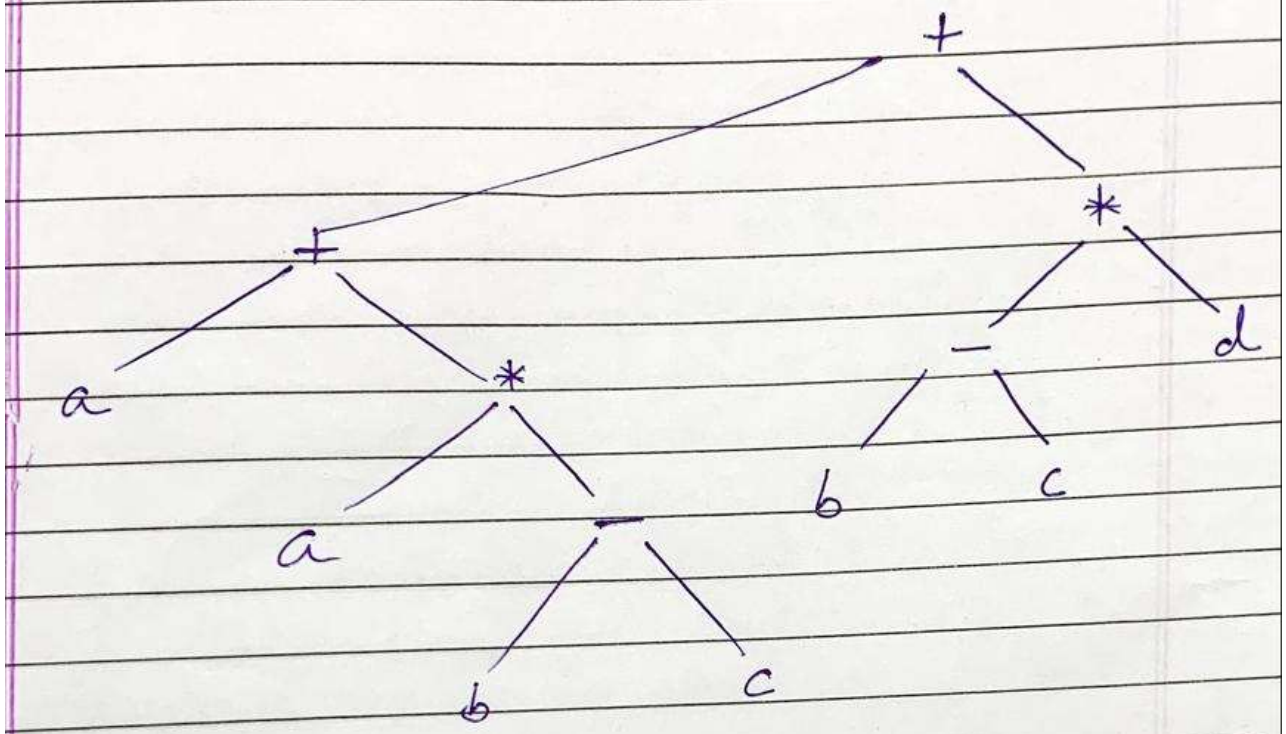
Triple

	OP	Arg1	Arg2
(0)	-	b	c
(1)	*	a	(0)
(2)	+	a	(1)
(3)	-	b	c
(4)	*	(3)	d
(5)	+	(2)	(4)

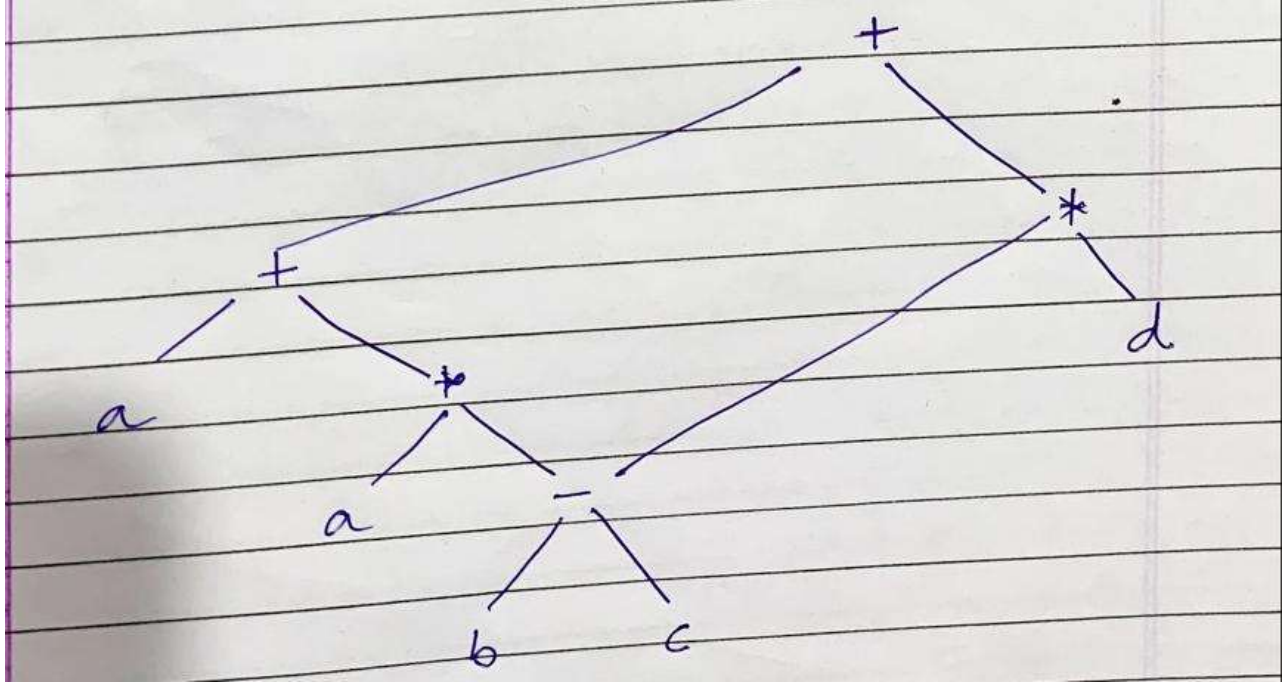
Indirect

(0)	(21)
(1)	(22)
(2)	(23)
(3)	(24)
(4)	(25)
(5)	(26)

Syntax tree



DAG: Directed Acyclic Graph



4. ans = a + b *c /2.0

TAC:

t1 = b*c
t2 = t1/2.0
t3 = a + t2
ans = t3

Quadruple

OP	Arg1	Arg 2	Result
*	b	c	t1
/	t1	2.0	t2
+	a	t2	t3
=	t3		ans

Triple

	OP	Arg 1	Arg 2
(0)	*	b	c
(1)	/	(0)	2.0
(2)	+	a	(1)
(3)	=	ans	(2)

Indirect Triple

(0)	(11)
(1)	(12)
(2)	(13)
(3)	(14)

5. $X = -a * b + -a * b$

TAC:

$t1 = -a$
$t2 = t1 * b$
$t3 = -a$
$t4 = t3 * b$
$t5 = t2 + t4$
$x = t5$

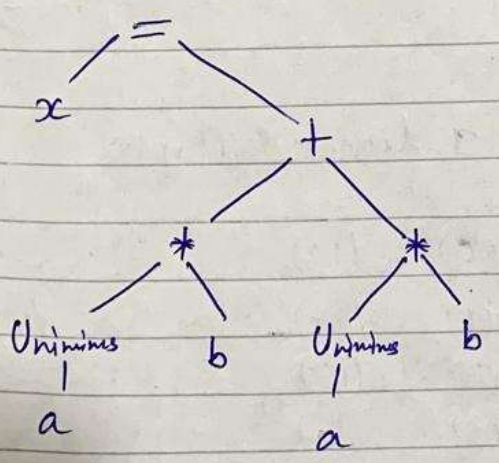
Quadruple:

OP	Arg 1	Arg 2	Result
Uminus	a		t1
*	t1	b	t2
Uminus	a		t3
*	t3	b	t4
+	t2	t4	t5
=	t5		x

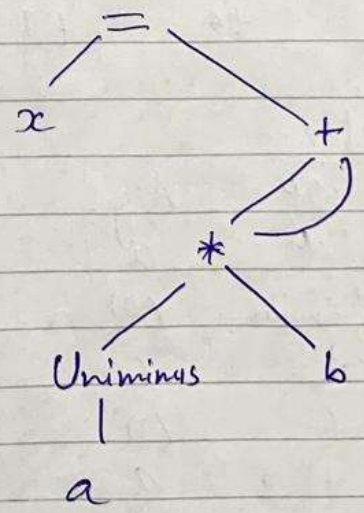
Triple:

	OP	Arg 1	Arg 2
(0)	Uminus	a	
(1)	*	(0)	b
(2)	Uminus	a	
(3)	*	(2)	b
(4)	+	(1)	(3)
(5)	=	x	(4)

AST:



DAG:



6. $-(a+b) + (c+d) - (a+b+c+d)$

TAC:

$t1 = a+b$
$t2 = \text{Uniminus}(t1)$
$t3 = c+d$
$t4 = t2 + t3$
$t5 = a+b$
$t6 = t5 + c$
$t7 = t6 + d$
$t8 = t4 - t7$

Quadruple:

OP	Arg 1	Arg 2	Result
*	a	b	t1
Uniminus	t1		t2
+	c	d	t3
+	t2	t3	t4
+	a	b	t5
+	t5	c	t6
+	t6	d	t7
−	t4	t7	t8

Triple:

	OP	Arg 1	Arg 2
(0)	*	a	b
(1)	Uniminus	(0)	
(2)	+	c	d
(3)	+	(1)	(2)
(4)	+	a	b
(5)	+	(4)	c
(6)	+	(5)	d
(7)	−	(3)	(6)

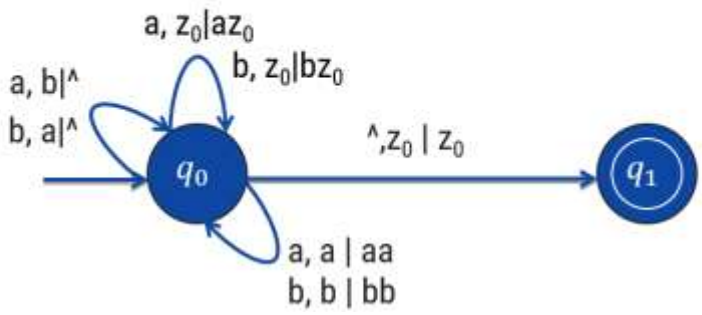
Indirect Triple

(0)	(13)
(1)	(14)
(2)	(15)
(3)	(16)
(4)	(17)
(5)	(18)
(6)	(19)
(7)	(20)

15. Design PDA to accept the following languages:

1. L = {x ∈ { a, b}* | $n_a(x) = n_b(x)$}, trace it for string "abbaba"

Move No	State	Input	Stack Symbol	Move(s)
1	q_0	a	z_0	(q_0, az_0)
2	q_0	b	z_0	(q_0, bz_0)
3	q_0	a	a	(q_0, aa)
4	q_0	b	b	(q_0, bb)
5	q_0	a	b	$(q_0, ^\wedge)$
6	q_0	b	a	$(q_0, ^\wedge)$
7	q_0	^	z_0	(q_1, z_0)
(all other combinations)			none	



trace the string "abbaba"

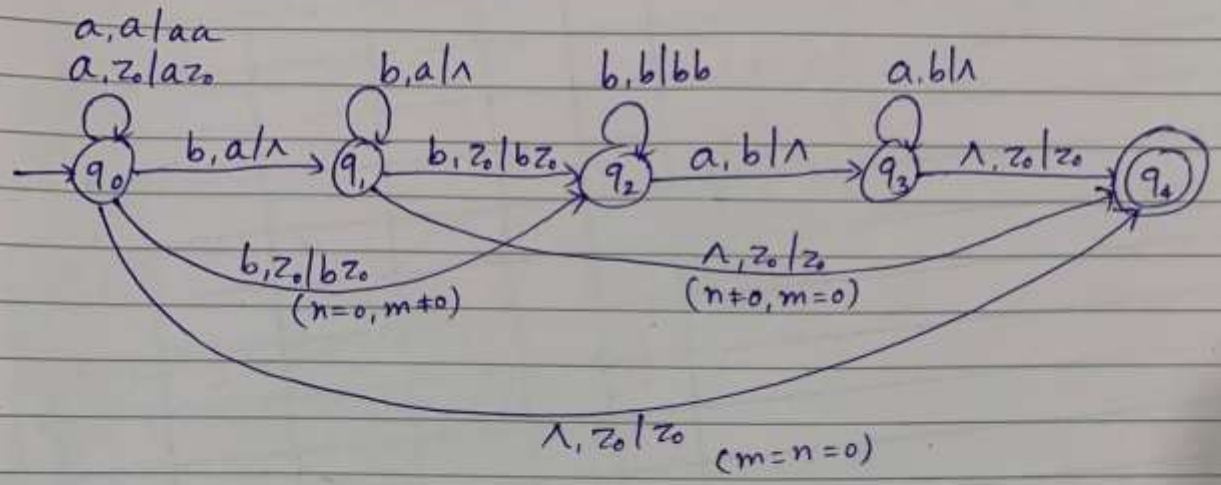
$(q_0, abbaba, z_0) \vdash (q_0, bbaba, az_0)$
$\vdash (q_0, baba, z_0)$
$\vdash (q_0, aba, bz_0)$
$\vdash (q_0, ba, z_0)$
$\vdash (q_0, a, bz_0)$
$\vdash (q_0, \Lambda, z_0)$
$\vdash (q_1, z_0)$

∴ string is valid.

2. L= { $a^n b^{n+m} a^m$ | n,m ≥ 0), trace it for string "abbbaa"

2. $L = \{a^n b^{n+m} a^m \mid n,m \geq 0\}$, trace it for "abbbaa".

$\therefore \underline{a^n b^n} \; \underline{b^m a^m}$



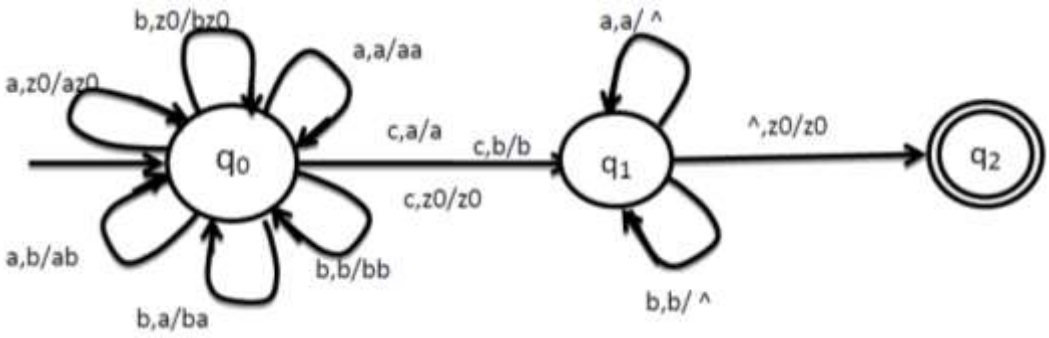
trace the string abbbaa

$(q_0, abbbaa, z_0) \vdash (q_0, bbbaa, az_0)$
$\vdash (q_1, bbaa, z_0)$
$\vdash (q_2, baa, bz_0)$
$\vdash (q_2, aa, bbz_0)$
$\vdash (q_3, a, b, z_0)$
$\vdash (q_3, \lambda, z_0)$
$\vdash (q_4, z_0)$

∴ String is valid.

3. L = { xcx^r / x Є {a,b}* }, trace it for string "abcba"

Move No.	State	i/p	Stack Symbol	Move
1	q_0	a	Z_0	(q_0,aZ_0)
2	q_0	b	Z_0	(q_0,bZ_0)
3	q_0	a	a	(q_0,aa)
4	q_0	b	a	(q_0,ba)
5	q_0	a	b	(q_0,ab)
6	q_0	b	b	(q_0,bb)
7	q_0	c	Z_0	(q_1, Z_0)
8	q_0	c	a	(q_1, a)
9	q_0	c	b	(q_1, b)
10	q_1	a	a	$(q_1, ^\wedge)$
11	q_1	b	b	$(q_1, ^\wedge)$
12	q_1	^	Z_0	(q_2, Z_0)

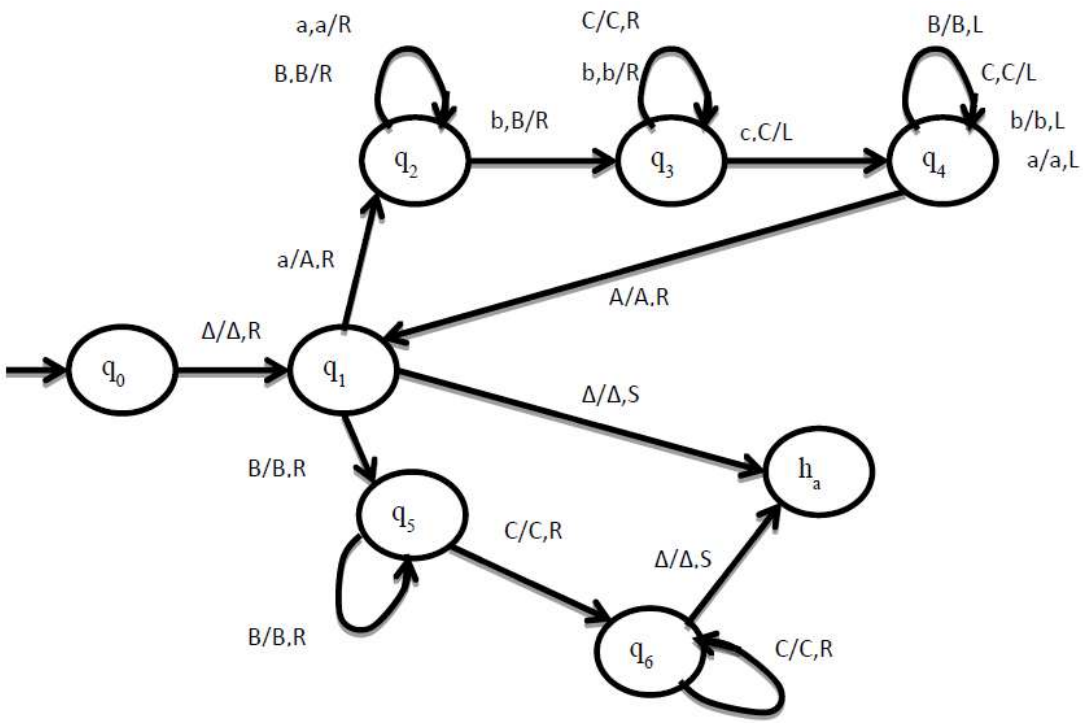
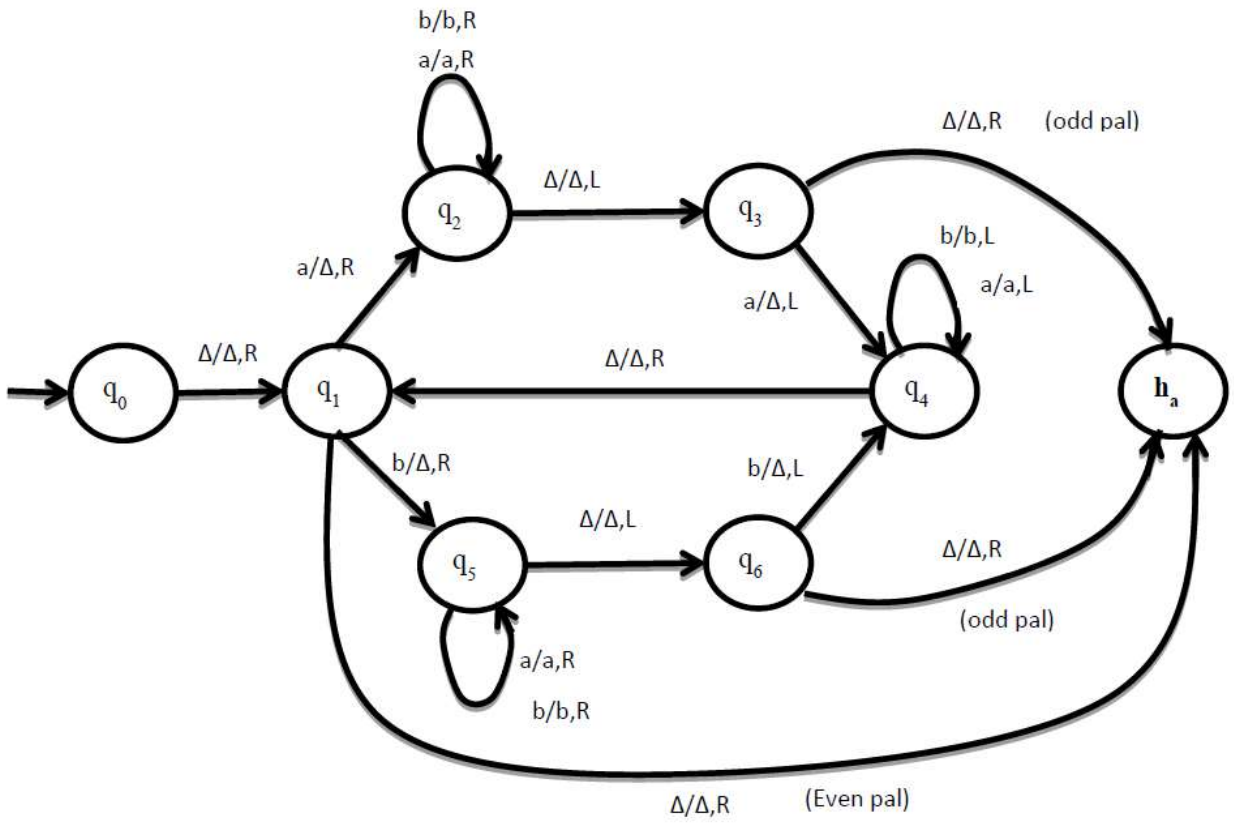


String:abcba

Move No.	Resulting State	unread i/p	Stack
Initial	q_0	abcba	Z_0
1	q_0	bcba	aZ_0
4	q_0	cba	baZ_0
9	q_1	ba	baZ_0
11	q_1	a	aZ_0
10	q_1	^	Z_0
12	q_2	^	Z_0

Design TM to accept the following languages:

1. L = { $a^n b^n c^n$ | n ≥ 0 }, trace it for string "aabbcc"



2. L = { xx^r / x Є {a,b}* }, trace it for string "ababa"